F25-147-D-Kairo

Project Team

Hafsa Imtiaz 22I-0959
Areen Zainab 22I-1115
Areeba Riaz 22I-1244

Session 2022-2026

Supervised by

Dr. Ali Zeeshan Ijaz

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2026

Contents

1 Introduction **1**
1.1 Existing Solutions . . . 2
1.2 Problem Statement . . . 4
1.3 Scope . . . 4
1.4 Modules . . . 5
1.4.1 Module 1: Meeting Bot Integration . . . 5
1.4.2 Module 2: Transcription and Summarization . . . 5
1.4.3 Module 3: Meeting Memory Engine . . . 6
1.4.4 Module 4: Task Board and Action Item Detection . . . 6
1.4.5 Module 5: Privacy, Control, and Analytics . . . 7
1.5 Work Division . . . 7
1.5.1 Gantt Chart . . . 8

2 Project Requirements **9**
2.1 Use-case/Event Response Table/Storyboarding . . . 9
2.2 Functional Requirements . . . 13
2.2.1 Module 1: Meeting Bot Integration & Workspace Management . . . 13
2.2.2 Module 2: Transcription and Summarization . . . 14
2.2.3 Module 3: Meeting Memory Engine & Knowledge Graph . . . 14
2.2.4 Module 4: Task Board & Action Item Management . . . 15
2.2.5 Module 5: Privacy, Control & Analytics . . . 16
2.3 Non-Functional Requirements . . . 16
2.3.1 Usability . . . 16
2.3.2 Performance . . . 17
2.3.3 Security . . . 17
2.3.4 Reliability . . . 18

3 System Overview **21**
3.1 Architectural Design . . . 21
3.1.1 Presentation Layer . . . 21
3.1.2 Application Layer . . . 22
3.1.3 Data Layer . . . 23
3.2 Data Design . . . 25

3.2.1 Data Storage Architecture 25
3.2.1.1 Storage Technologies 25
3.2.1.2 Data Organization Layers 25
3.2.2 Major Data Entities 26
3.2.3 Data Transformation and Processing Pipeline 28
3.2.4 Data Storage Items 28
3.2.5 Summary 29
3.3 Domain Model 29
3.4 Design Models 31
3.4.1 Class Diagram 31
3.4.2 Activity Diagram 31
3.4.3 Class-Level Sequence Diagram 33
3.4.4 State-Transition Diagram 37

4 Implementation and Testing **39**
4.1 Algorithm Design 40
4.1.1 User Sign Up and Login Algorithm 41
4.1.2 Workspace Creation and Member Management Algorithm 42
4.1.3 Meeting Scheduling Algorithm 43
4.1.4 Bot Meeting Joining Algorithm 44
4.1.5 Real-Time Transcription Processing Algorithm 45
4.1.6 Speaker Diarization Algorithm 46
4.1.7 Action Item Extraction Algorithm 47
4.1.8 Meeting Summary Generation Algorithm 48
4.1.9 Whisper Mode Micro-Recap Algorithm 49
4.1.10 Post-Meeting Processing Pipeline 50
4.1.11 Meeting Memory Graph Generation Algorithm 51
4.1.12 Semantic Search Algorithm 53
4.1.13 Privacy Mode Algorithm 53
4.1.14 Calendar Integration Algorithm 55
4.2 External APIs/SDKs 56
4.3 Testing Details 58
4.3.1 Unit Testing 58
4.3.1.1 Authentication Module Test Cases 58
4.3.1.2 Meeting Management Test Cases 61
4.3.1.3 Transcription Service Test Cases 64
4.3.1.4 Action Item Extraction Test Cases 67
4.3.1.5 Whisper Mode Test Cases 69
4.3.1.6 Meeting Memory Graph Test Cases 71
4.3.1.7 Semantic Search Test Cases 73

4.3.2 Integration Testing 77
4.3.2.1 End-to-End Workflow Test 77
4.3.3 Testing Statistics Overview 78

5 Conclusions and Future Work **81**
5.1 Conclusion 81
5.2 Future Work 81

References **84**

List of Figures

1.1 Gantt Chart 8

2.1 Use-Case Diagram 13

3.1 Simple System Box and Line Diagram 23
3.2 3-Tier System Architectural Design Diagram 24
3.3 Entity-Relationship Diagram of Kairo's Data Model 26
3.4 Domain Model Diagram of the System 30
3.5 Class diagram 31
3.6 Activity Diagram of Kairo showing the meeting lifecycle. 32
3.7 Class-level Sequence Diagram for the registration module, highlighting the interactions between user, authentication service, and workspace components. 33
3.8 Class-level Sequence Diagram illustrating both automatic and manual bot joining processes, followed by transcription initiation. 33
3.9 Class-level Sequence Diagram for visualizing the Knowledge Graph. . . . 34
3.10 Class-Level Sequence Diagram of Live Transcription 34
3.11 Class-level Sequence Diagram illustrating action item detection during meetings. 35
3.12 Class-level Sequence Diagram for meeting summary generation. 35
3.13 Class-level Sequence Diagram depicting Kanban generation within Kairo's task management module. 36
3.14 Class-level Sequence Diagram depicting the retrieval process from Kairo's Meeting Memory using conversational queries and semantic vector search. 36
3.15 Class-level Sequence Diagram illustrating the activation and handling of Privacy Mode in Kairo. 37
3.16 State-Transition Diagram of Kairo showing the end-to-end meeting lifecycle. 38

4.1 Complete Mocha/Chai Test Suite Results (159 Tests) 79

List of Tables

1.1 Comparison of Existing Solutions 3
1.2 Work Breakdown 7

2.1 Use Cases for Kairo 10
2.1 Use Cases for Kairo 11
2.1 Use Cases for Kairo 12

3.1 Key Entities and Their Relationships 27

4.1 Summary of Algorithms Implemented in Kairo 40
4.2 External APIs and SDKs Used in Kairo Implementation 57
4.3 Authentication Module Test Cases 61
4.4 Meeting Management Test Cases 64
4.5 Transcription Service Test Cases 66
4.6 Action Item Extraction Test Cases 69
4.7 Whisper Mode Test Cases 71
4.8 Meeting Memory Graph Test Cases 73
4.9 Semantic Search Test Cases 76
4.10 Integration Test Case: Complete Meeting Lifecycle 78
4.11 Testing Statistics Overview — Kairo FYP (Complete Development) . . . 78

Chapter 1

Introduction

This document presents the completion of all four iterations and the entire FYP. Building upon the intelligent, context-aware foundation established in Iterations #1 and #2, this report covers the advanced feature development and system maturation achieved through Iterations #3 and #4, culminating in a feature-complete, production-ready meeting intelligence platform. Together, these final iterations introduced capabilities that differentiate Kairo through enhanced understanding, deeper contextualization, enhanced visualization of meeting knowledge, and seamless calendar integration.

Meetings remain central to organizational coordination, yet most existing tools primarily facilitate communication without effectively capturing or operationalizing the rich context generated within them. Platforms such as Zoom and Google Meet offer recording and transcription features, but still fall short in supporting continuity, contextual awareness, and actionable insights. Prior research highlights persistent challenges in areas such as action-item extraction, context modeling, and cross-meeting knowledge retention [8; 11]. These limitations become more pronounced in remote and hybrid environments, where fragmented discussions and lack of structured memory hinder productivity.

To address these challenges, Iterations #3 and #4 introduce a comprehensive set of advanced "wow" features that transform Kairo into a fully integrated meeting intelligence system. Iteration #3 focuses on the development of a Meeting Memory Graph for visualizing relationships between discussions, decisions, and participants, task contextual micro-channels for focused collaboration, and an analytics dashboard providing insights such as key performance indicators, participation patterns, and discussion trends. Iteration #4 extends these capabilities with calendar integration for automated meeting scheduling and synchronization, and deployment strategies further enhancing user experience and system automation. Together, these enhancements elevate Kairo from a transcription and task management tool to a fully integrated platform for knowledge-driven collaboration, aligning with emerging research systems such as MeetMap and AreWeOnTrack [3; 4].

1.1 Existing Solutions

Several meeting intelligence tools and research systems provide transcription, summarization, and analytics capabilities. However, most fall short in long-term context preservation and cross-meeting memory construction. This analysis examines existing solutions to highlight the gap Kairo addresses: creating a persistent, context-aware meeting memory system that links discussions, decisions, and tasks across sessions.

Commercial Meeting Intelligence Tools

Fireflies.ai [7] offers automated transcription, topic summaries, and basic action item extraction with integration across popular conferencing platforms. However, it lacks semantic continuity across meetings and graph-based representation of discussions.

Otter.ai [15] excels at real-time transcription with speaker identification and keyword search, yet falls short when it comes to extracting tasks or maintaining memory across sessions that connects discussions over time.

Fathom AI [6] focuses on recording, generating summaries, and creating highlight clips primarily for sales workflows. While effective for its target use case, it lacks cross-meeting linking, real-time action item modeling, and contextual recall capabilities.

MeetGeek [12] provides detailed summaries and analytics dashboards with keyword-based insights. However, these summaries remain static without knowledge graph construction or context-aware retrieval of past discussions.

Krisp AI [10] integrates transcription with noise cancellation and speaker analytics. While useful for meeting metrics, it does not build long-term memory models or support retrieval of previous decisions.

Motion AI [13] converts meeting summaries into scheduled tasks on user calendars. Its focus on productivity scheduling comes without deep meeting understanding, semantic modeling, or cross-meeting knowledge tracking.

Research Prototypes

MeetMap [4] uses LLMs to map topic transitions and generate structured meeting flow diagrams. It provides context-aware analysis within single meetings but does not persist context across sessions or extract actionable tasks.

AreWeOnTrack [3] evaluates progress toward predefined project goals by analyzing conversations. It lacks transcript generation, knowledge graphs, and long-term memory structures, operating only within single sessions.

These systems demonstrate progress in transcription accuracy and summarization quality but consistently lack *persistent memory*, *cross-meeting semantic linking*, and *graph-based understanding of organizational discussions*. This gap motivates Kairo's development, integrating real-time meeting capture with a Meeting Memory Engine that links topics, tasks, and decisions across multiple sessions.

Table 1.1: Comparison of Existing Solutions

System	**Capabilities**	**Limitations**
Fireflies.ai [7]	Automated transcription, keyword tagging, searchable summaries with platform integration	No cross-meeting memory, graph-based linking, or reliable action item modeling
Otter.ai [15]	Real-time transcription, speaker identification, keyword search	No task extraction, session continuity, or graph-based representation
Fathom AI [6]	Recording, summaries, highlight clips, CRM integration	Sales-focused; no semantic recall, multi-meeting linking, or knowledge graphs
MeetGeek [12]	Detailed summaries, insights, analytics dashboards	Isolated per-meeting summaries; no retrieval engine or long-term memory
Krisp AI [10]	Transcription with noise cancellation and meeting analytics	No semantic modeling, memory systems, or action item detection
Motion AI [13]	Converts notes to tasks and auto-schedules them	No transcript analysis, semantic modeling, or cross-meeting memory
MeetMap [4]	LLM-based topic mapping and structured flow visualization	Single-meeting scope; no persistent memory, task extraction, or cross-session retrieval
AreWeOnTrack [3]	Conversation analysis for goal alignment	No transcription, knowledge graphs, action items, or cross-meeting continuity

While these systems have advanced meeting capture capabilities, they leave fundamental organizational challenges unaddressed.

1.2 Problem Statement

Meetings are central to teamwork, yet organizations often struggle to turn them into clear and actionable outcomes. Decisions, tasks, and responsibilities are frequently undocumented or scattered across emails, chat logs, or lengthy recordings, making them difficult to retrieve when needed. This lack of continuity leads to repeated conversations, wasted time, and delays in project progress. Without connected memory across meetings, knowledge remains fragmented, preventing teams from building on past discussions.

Existing tools such as transcription services or note-taking apps offer only partial solutions, as they capture information but fail to ensure accountability or structured task management. Most systems lack real-time support and produce summaries only after meetings end, offering little help in fast-paced settings like agile stand-ups or sprint planning. Recent research such as MeetMap [4] has demonstrated the potential of LLMs for real-time dialogue mapping, yet such systems remain prototypes without long-term memory or cross-meeting task tracking.

This project aims to address these challenges by developing Kairo, an intelligent meeting platform that transforms conversations into structured, actionable knowledge. Kairo integrates real-time transcription, automated task extraction, and a persistent Meeting Memory Engine that maintains context across sessions. By connecting discussions, decisions, and action items into a unified knowledge graph, Kairo enables teams to retrieve past insights, track task evolution, and maintain continuity across organizational meetings.

1.3 Scope

The scope of Kairo is to transform corporate meetings into structured, trackable, and actionable outcomes. The system captures conversations, generates accurate transcriptions, and ensures continuity through a **Meeting Memory Engine** that recalls past discussions and links unresolved items across sessions. It extracts action items, assigns deadlines and responsibilities, and organizes tasks on a Kanban-style task board. Post-meeting, calendar-based reminders and analytics dashboards further enhance accountability and productivity. Kairo integrates with platforms like Zoom and Google Meet as a bot participant and provides users with dashboards for managing tasks, viewing summaries, and receiving reminders. To support its memory graph and semantic retrieval capabilities, Kairo leverages FAISS for vector similarity search and Neo4j for graph-based contextual retrieval [14]. PostgreSQL serves as the primary data store due to its scalability and compatibility with AI-driven workloads [5; 18].

The system focuses strictly on corporate meeting intelligence, excluding broader organizational management features. Comprehensive project management features such as

HR-related functionalities, or financial tracking, etc., fall outside the project scope and are excluded. Advanced features like real-time video analysis, emotion detection from facial expressions, automated meeting scheduling, and offline meeting capture are not part of this project. Kairo is also limited to Zoom and Google Meet; thereby, it does not support any integration with other enterprise communication platforms.

1.4 Modules

The system is divided into modules to ensure a modular and structured implementation. Each module focuses on specific functionalities of the system.

1.4.1 Module 1: Meeting Bot Integration

Kairo joins third-party meeting platforms such as Zoom or Google Meet as a participant, recording audio in real time and preparing input for transcription.

1. Auto-Join & Capture: Automatically joins scheduled meetings (via calendar integrations), records audio, and starts real-time transcription.
2. Calendar Integrations: Sync with Google/Outlook calendars to auto-detect meetings.
3. Third-Party Tool Integrations: Connect to Jira, Trello, Slack, and Google Drive for seamless push/pull of meeting notes and tasks.
4. Manage session-level authentication and permissions.

This functionality is conceptually aligned with Recall.ai's approach to meeting bot automation [16], enabling autonomous participation across digital meeting platforms.

1.4.2 Module 2: Transcription and Summarization

Audio streams are converted into accurate text with automated speaker identification, then processed into tiered summaries for different stakeholder needs.

1. Real-Time Transcription (WhisperX-based): Produces accurate transcripts with speaker diarization using Pyannote and Resemblyzer.
2. Summarization Engine: Generates layered summaries (high-level for executives, detailed for participants).

3. Whisper Mode: Provides real-time micro-recaps every 5–10 minutes, displayed in chat/UI so late joiners instantly catch up.
4. Highlight unresolved items from previous meetings using the memory engine.

The transcription layer is built upon WhisperX [1] for high-fidelity speech recognition, while Diart [9] and Resemblyzer [17] provide real-time diarization and speaker embedding support.

1.4.3 Module 3: Meeting Memory Engine

The core of Kairo's intelligence lies in maintaining continuity across meetings by storing past discussions and surfacing relevant context during future sessions.

1. Maintain a structured memory of past meeting outcomes.
2. Meeting Memory Graph: Visualize concepts, tasks, and decisions across sessions in an interactive Neo4j-style graph.
3. Link unresolved tasks, emphasize pending discussions, and clearly outline dependencies between action items, discussions, and meetings.
4. Provide contextual continuity across sessions.

The memory engine leverages vector embeddings and graph relationships inspired by recent advancements in conversational memory and retrieval frameworks [2; 14].

1.4.4 Module 4: Task Board and Action Item Detection

Action items and commitments are automatically identified from conversations and organized into a collaborative task management interface.

1. Action Item Detection: Automatically extracts tasks, commitments, and deadlines from conversations.
2. Kanban Board Integration: Action items flow directly into Kanban-style boards inside Kairo.
3. Task Contextual Micro-Channels: Each task has a "Meeting Context" tab, collecting discussions, snippets, and decisions tied to it across multiple meetings.
4. Allow updates and edits by team members.

Kairo's action item pipeline is informed by action item detection models discussed in [11] and extended with contextual modeling for better precision.

1.4.5 Module 5: Privacy, Control, and Analytics

Accountability, compliance, and performance insights are managed through granular privacy controls and comprehensive analytics.

1. Privacy & Compliance Mode: Toggle transcription/recording off for sensitive parts while still logging metadata, managed by role-based permissions.
2. Role-Based Access Control: Restricts visibility of sensitive meeting data, with different levels of detail for executives, managers, and contributors.
3. Smart Search & Speaker Recall: Retrieve past meeting content by keyword, task, decision, or speaker (e.g., "What did Sarah say about X?") using semantic search.
4. Send follow-up reminders through calendar integration.
5. Provide analytics dashboards for productivity and meeting efficiency.

1.5 Work Division

Responsibilities have been divided among team members based on modules and features.

Table 1.2: Work Breakdown

Members	**Division**
Hafsa Imtiaz	Workspace Management, Meeting Memory Engine, Smart Search, Transcription Timeline, Multimodal Meeting Capture, Privacy Mode
Areeba Riaz	Bot-Based Meeting Participation, Meeting Memory Engine, Action Item Detection, Follow-Up Reminders, Deployment, UI/UX
Areen Zainab	UI/UX, Summarization Engine, Meeting Memory Engine, Smart Search, Meeting Memory Graph, Calendar Integrations

1.5.1 Gantt Chart

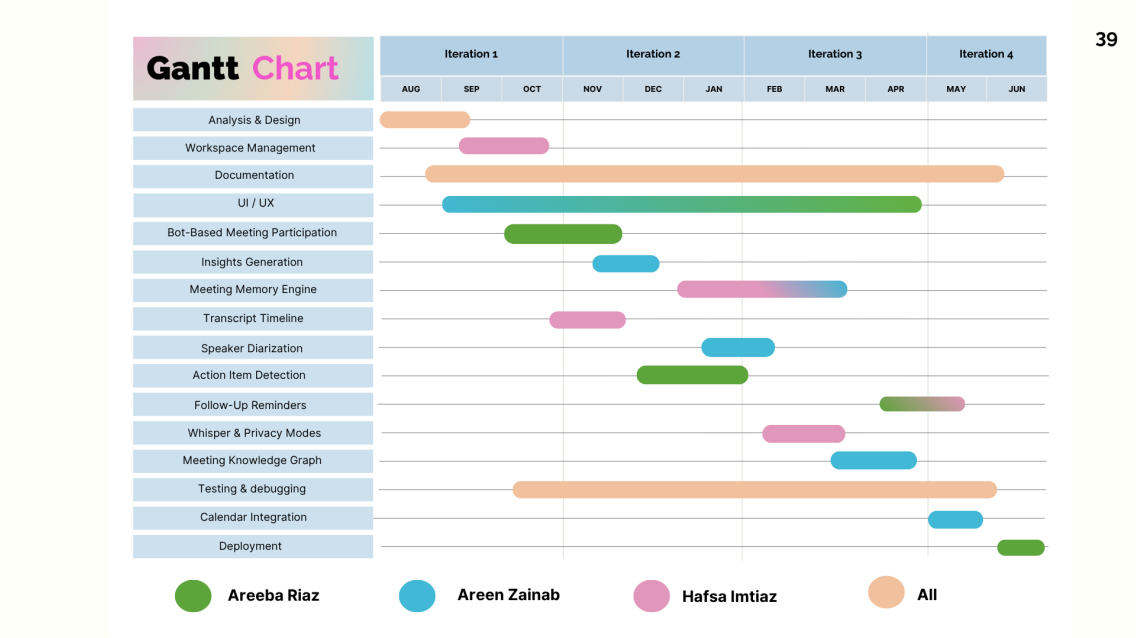

Figure 1.1: Gantt Chart

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case/Event Response Table/Storyboarding

Since **Kairo** is an interactive end-user application with multiple actors (team members, managers, system components, and external services), the most suitable requirement representation technique is a **Use Case Model**. The following table presents the primary use cases initially identified during **Iteration #1** and refined through all subsequent iterations. These capture the main interactions between users and the system as evolved across the complete FYP development.

Title	Actor(s)	Preconditions	Postconditions	Main Flow
Manage Workspace & User Roles	Team Lead / Manager	User must be registered and logged in; a workspace must exist or be created.	The workspace is created or updated, and user roles and permissions are stored in the system.	Manager enters workspace details; system validates input and creates workspace. Then, in User Management, the manager adds or removes members, assigns roles, updates permissions, and saves the configuration.
Connect Calendar Service	Team Lead / Manager / Employee	User is a workspace member.	Calendar is linked and synchronized.	User authorizes Google or Outlook integration; upon approval, the system retrieves upcoming meetings.
Join Meeting (Auto / Manual)	Kairo Bot / Team Lead / Manager	A valid meeting link or event must exist.	Bot joins and begins capture.	System auto-joins at the scheduled time or on manual user request.
Real-Time Transcription	Kairo Bot	Bot must be connected to the meeting.	Audio streams sent for transcription.	Bot continuously captures audio and forwards it to the transcription module (WhisperX) for live text output.

Table 2.1: Use Cases for Kairo

Perform Speaker Diarization	System	Transcript must exist.	Text segments are tagged with speaker identity.	System performs voice-based diarization and updates the transcript with labeled segments.
Generate Whisper Mode Micro-Recaps	System	Live transcription is active.	Micro-recaps are displayed.	System generates 5–10 minute summaries and presents them in the meeting UI for context and late joiners.
Generate Post-Meeting Summary	System	Meeting has ended, and a transcript exists.	Summary is generated and stored.	System analyzes transcripts, decisions, and key points to generate layered summaries.
Detect Action Items	System	Processed transcript must exist.	Action items extracted and saved.	System identifies and stores commitments, tasks, and deadlines from the discussion.
Auto-Populate Kanban Board	System	Tasks must be finalized.	Kanban board is populated.	System converts action items into structured Kanban tasks organized by status.

Table 2.1: Use Cases for Kairo

Manage Task	Employee / Manager	Tasks exist on the Kanban board.	Task details updated.	User selects a task, reviews context, updates status or notes, and saves changes; system synchronizes updates and context via the Memory Engine.
Update Meeting Memory & Knowledge Graph	System	Summaries, transcripts, and tasks must be finalized.	Knowledge Graph and meeting memory updated.	System extracts structured data, links related entities, and updates semantic relationships across meetings.
Intelligent Search & Retrieval	Employee / Manager	Meeting and memory data must exist.	Search results retrieved and displayed.	User submits query; system performs contextual, semantic, and speaker-based retrieval to return relevant results.
Activate Privacy & Compliance Mode	Team Lead	Meeting must be in progress.	Sensitive segments excluded from recording.	Team Lead activates Privacy Mode; system pauses recording, logs duration and reason, and resumes upon reactivation.

Table 2.1: Use Cases for Kairo

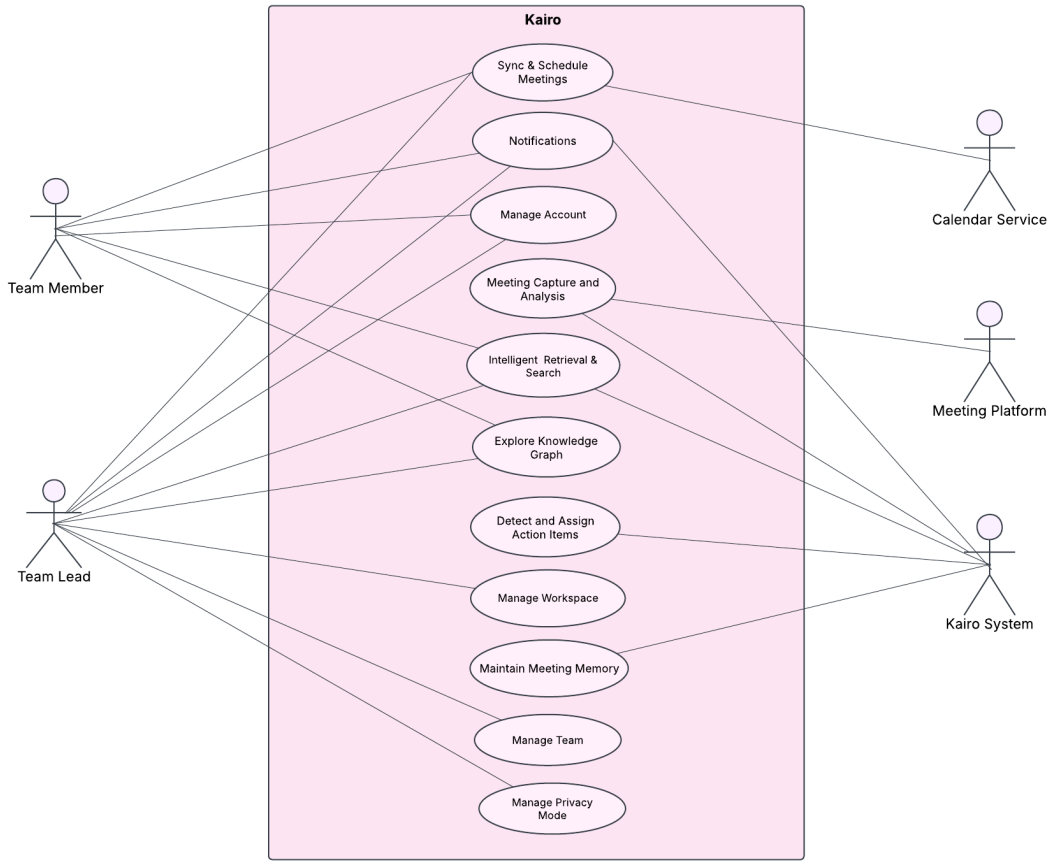

Figure 2.1: Use-Case Diagram

2.2 Functional Requirements

This section describes the functional requirements of the Kairo system, expressed in natural language and organized by system modules. Each module corresponds to a major functional area of the system and includes specific requirements that describe the expected behavior and interactions.

2.2.1 Module 1: Meeting Bot Integration & Workspace Management

This module handles workspace creation, user access, meeting participation, calendar integration, and external tool connections.

1. The system shall allow users to create and join workspaces, and administrators to invite members and assign roles (Admin, Manager, Employee).

2. The system shall enforce secure authentication and role-based access control (RBAC), restricting meeting data visibility and feature access based on user roles.
3. The system shall integrate with Google Calendars to detect upcoming meetings and automatically join supported platforms (e.g., Google Meet, Zoom) as a silent bot participant.
4. The system shall capture audio, video, and screen shares during meetings, and allow manual capture of visuals (e.g., slides, charts, whiteboards).
5. The system shall timestamp and synchronize captured visuals with transcripts and store them securely.
6. The system shall provide pause and resume controls for recording and transcription through Privacy Mode.

2.2.2 Module 2: Transcription and Summarization

This module is responsible for converting speech into text, generating summaries, and providing micro-recaps.

1. The system shall provide real-time transcription of speech into text and store transcripts linked to their corresponding meeting records.
2. The system shall assign speaker identities using diarization and voice embeddings.
3. The system shall generate layered summaries at both executive and detailed levels, including agenda recaps, key discussion points, decisions, and action items.
4. The system shall allow the meeting host to edit, enhance, or finalize the generated summaries.
5. The system shall generate real-time micro-recaps every 5–10 minutes, display them in the chat/UI ticker, and allow late joiners to view past recaps.
6. The system shall provide a collaborative note-taking panel during meetings, where notes are timestamped, attributed to authors, synchronized in real-time, and included in the post-meeting summary.

2.2.3 Module 3: Meeting Memory Engine & Knowledge Graph

This module maintains long-term knowledge continuity, retrieves past meeting context, and visually links related information.

1. The system shall maintain long-term memory of transcripts, summaries, tasks, and decisions.

2. The system shall retrieve contextually relevant past discussions when a new meeting begins.

3. The system shall display unresolved tasks and recurring topics in a Context Panel.

4. The system shall maintain an interactive, visual knowledge graph linking topics, people, tasks, and decisions.

5. The system shall provide an interactive transcript timeline with timestamps.

6. The system shall highlight decisions, tasks, and speaker annotations in transcripts.

7. The system shall allow users to filter and search transcripts by speaker, topic, or task, and jump to specific timestamps.

2.2.4 Module 4: Task Board & Action Item Management

This module extracts tasks from conversations and organizes them into Kanban workflows.

1. The system shall monitor live transcripts to detect task-related commitments, link each task to the corresponding speaker and role, and extract deadlines from natural language expressions.

2. The system shall display identified tasks in a live confirmation panel where users can review, edit, confirm, or discard them.

3. The system shall convert confirmed action items into structured tasks containing a description, assignee, type, priority, and deadline, and display them on a Kanban-style task board.

4. The system shall allow users to manage tasks on the Kanban board, including state transitions (To-Do $\rightarrow$ In Progress $\rightarrow$ Done), metadata (assignee, deadline, priority), and real-time drag-and-drop updates.

5. The system shall create a dedicated micro-channel for each task to support ongoing discussions and aggregate related transcripts, notes, and references within that channel.

2.2.5 Module 5: Privacy, Control & Analytics

This module focuses on compliance, advanced search, reminders, and performance analytics.

1. The system shall provide a Privacy Mode that allows users to pause transcription and recording while still logging meeting metadata such as duration and participant information.
2. The system shall restrict Privacy Mode controls to users with the required role permissions.
3. The system shall provide an advanced search capability that allows users to retrieve past meetings by keyword, task, decision, concept, or speaker-specific queries (e.g., "What did Sarah say about deadlines?").
4. The system shall support semantic search using embeddings for contextual understanding.
5. The system shall auto-sync tasks and deadlines extracted from meetings with integrated calendar platforms and send reminders via calendar notifications and in-app alerts, including task details, due dates, and meeting context.
6. The system shall offer an analytics dashboard that summarizes team performance and meeting outcomes through metrics such as task completion rate, meeting duration trends, participation frequency, and unresolved tasks.
7. The system shall allow users to filter analytics data by timeframe, project, or user for more targeted insights.

2.3 Non-Functional Requirements

This section specifies the non-functional requirements for the Kairo system. These quality requirements define the key performance, usability, security, and reliability characteristics that ensure Kairo operates efficiently and consistently. Each requirement is specific, quantitative, and verifiable.

2.3.1 Usability

These requirements ensure that Kairo is intuitive, learnable, and efficient for diverse user roles (executives, managers, employees). They address ease of learning, ease of use, error avoidance and recovery, the efficiency of interactions, and accessibility.

ID	Requirement	Description & Measurement
USE-1	Ease of Onboarding	New users shall be able to join a workspace and set up an auto-join meeting within 10 minutes without external support.
USE-2	Micro-Recap Accessibility	Late joiners shall access all past micro-recaps within 5 seconds of joining the meeting.
USE-3	Error Recovery	Users shall be able to undo or edit incorrect summaries or extracted tasks within 2 interactions.
USE-4	Accessibility Compliance	The user interface shall provide proper color contrast, keyboard navigation, and screen reader support for accessibility.
USE-5	Multimodal Navigation	Users shall retrieve meeting information via text query, visual graph exploration, or timeline navigation without requiring technical knowledge.

2.3.2 Performance

These requirements ensure that Kairo operates with real-time responsiveness and scalability, particularly during transcription, summarization, and retrieval tasks. Performance requirements specify response times and processing speeds for various system operations.

ID	Requirement	Description & Measurement
PER-1	Transcription Latency	Real-time transcription shall display text within 2 seconds of speech.
PER-2	Summarization Speed	Post-meeting summaries shall be generated within 60 seconds for a 60-minute meeting.
PER-3	Micro-Recap Interval	Whisper Mode micro-recaps shall appear within 30 seconds of their scheduled 5–10 minute interval.
PER-4	Search Query Response Time	Semantic search queries (including speaker-based queries) shall return results within 5 seconds.
PER-5	Auto-Join Startup Time	Auto-join shall initiate and connect to a scheduled meeting within 15 seconds of the start time.

2.3.3 Security

These requirements ensure confidentiality, integrity, and controlled access to sensitive meeting data. Security requirements address authentication, authorization, data protec-

tion, audit trails, and compliance with privacy regulations.

ID	Requirement	Description & Measurement
SEC-1	Access Control Enforcement	Role-based access control (RBAC) shall ensure 100% prevention of unauthorized access to restricted transcripts, tasks, and graphs.
SEC-2	Privacy Mode Compliance	When Privacy Mode is activated, no audio/video/text data shall be captured — only metadata — ensuring zero leakage of sensitive content.
SEC-3	Intrusion Detection & Logging	The system shall detect unauthorized access attempts and log them within 10 seconds, generating alerts to administrators.
SEC-4	Audit Trail Availability	A complete audit trail (meeting joins, edits, task changes) shall be maintained and retained for at least 1 year.
SEC-5	External API Security	All third-party integrations shall use OAuth 2.0 or equivalent secure protocols and require explicit user consent before data sharing.

2.3.4 Reliability

These requirements ensure that Kairo consistently performs its intended functions — capturing, transcribing, summarizing, and storing meeting intelligence — with minimal failure. Reliability requirements address system availability, fault tolerance, data persistence, and recovery mechanisms.

ID	Requirement	Description & Measurement
REL-1	System Uptime	The system shall have an uptime of $\geq 99.5\%$ per month, excluding scheduled maintenance.
REL-2	Auto-Recovery from Failures	In case of a failure during a meeting (e.g., transcription crash), the system shall auto-recover and resume within 60 seconds without data loss.
REL-3	Data Persistence	To prevent data loss during unexpected outages, all meeting data (transcripts, summaries, tasks) shall be persisted within 5 seconds of generation.
REL-4	Graceful Degradation	If any external integration (e.g., Calendar API) fails, the system shall continue core operations (capture, transcription, summarization) without complete shutdown.
REL-5	Failure Logging & Alerts	Any failure in auto-join, capture, or summarization modules shall trigger real-time error logs and alerts to system admins within 30 seconds.

Chapter 3

System Overview

This chapter presents the comprehensive design and architecture of the Kairo intelligent meeting collaboration platform. It establishes the technical foundation necessary for understanding how the system transforms meeting-related information into actionable intelligence while maintaining scalability, modularity, and efficient data management.

3.1 Architectural Design

Kairo follows a robust **three-tier architecture** designed to ensure scalability, modularity, and a clear separation of concerns.

The three primary tiers are the **Presentation Layer**, the **Application Layer**, and the **Data Layer**. This structure enhances scalability and maintainability; and also enables independent evolution of Kairo's intelligence capabilities without disrupting the broader system architecture.

3.1.1 Presentation Layer

The Presentation Layer forms the interface between users and the system. It is where users interact with Kairo through intuitive web-based tools:

- **Web Dashboard:** Provides access to meeting data, insights, and team performance metrics.
- **Kanban Board:** Visualizes and manages action items and tasks automatically generated from meeting outcomes.
- **Search & Graph Explorer:** Enables semantic search and exploration of relationships within the Knowledge Graph.

This layer remains stateless and communicates with backend services through the **API Gateway**, ensuring efficient, secure, and device-agnostic access.

3.1.2 Application Layer

The Application Layer forms the logical and functional backbone of Kairo. It manages all business logic, workflow orchestration, and communication between the other layers and workspace & contextual subsystems, ensuring everything remains synchronized.

Its key responsibilities include:

- **System Coordination:** Orchestrates the end-to-end meeting lifecycle—from scheduling to post-meeting insight delivery.
- **Integration Management:** Connects with external services such as Google Calendar and Zoom to synchronize meetings and automate recording ingestion.
- **User & Workspace Handling:** Manages authentication, authorization, and workspace configurations for teams.
- **Task & Notification Control:** Automates task generation from meeting outcomes and sends relevant summaries or alerts to users.
- **API Gateway:** Acts as a secure entry point for frontend requests and routes them to respective backend microservices.

Kairo Intelligence Core (Logical AI Subsystem) Nested within the Application Layer is the **Kairo Intelligence Core**, which encapsulates all context-driven functionality. It serves as the analytical engine that transforms unstructured meeting data into actionable intelligence. The subsystem includes:

- **Transcription & Summarization:** Converts meeting audio into text and generates concise, structured summaries.
- **Action Item Extraction:** Identifies key decisions and tasks discussed during meetings.
- **Contextual NLP & Semantic Embedding:** Interprets meaning, relationships, and context, enabling intelligent search and retrieval.
- **Knowledge Graph & Insights:** Builds connected representations of people, topics, and outcomes to power analytics and knowledge discovery.

By consolidating these capabilities into a modular subsystem, the Intelligence Core allows Kairo to evolve its contextual abilities independently of other services while maintaining scalability and robust system performance.

3.1.3 Data Layer

The Data Layer is responsible for all persistent storage, indexing, and retrieval operations in Kairo. It comprises multiple specialized databases, each optimized for specific data types. The Workspace DB stores team and user configurations, while the Meeting DB retains transcripts, metadata, and summaries. The Knowledge Graph DB manages relationships and entities extracted by the AI subsystem. The Vector DB (pgVector) stores semantic embeddings for fast similarity search and contextual retrieval. A dedicated Storage module also handles raw meeting recordings, visuals, and supporting documents, ensuring efficient and organized data management across the system.

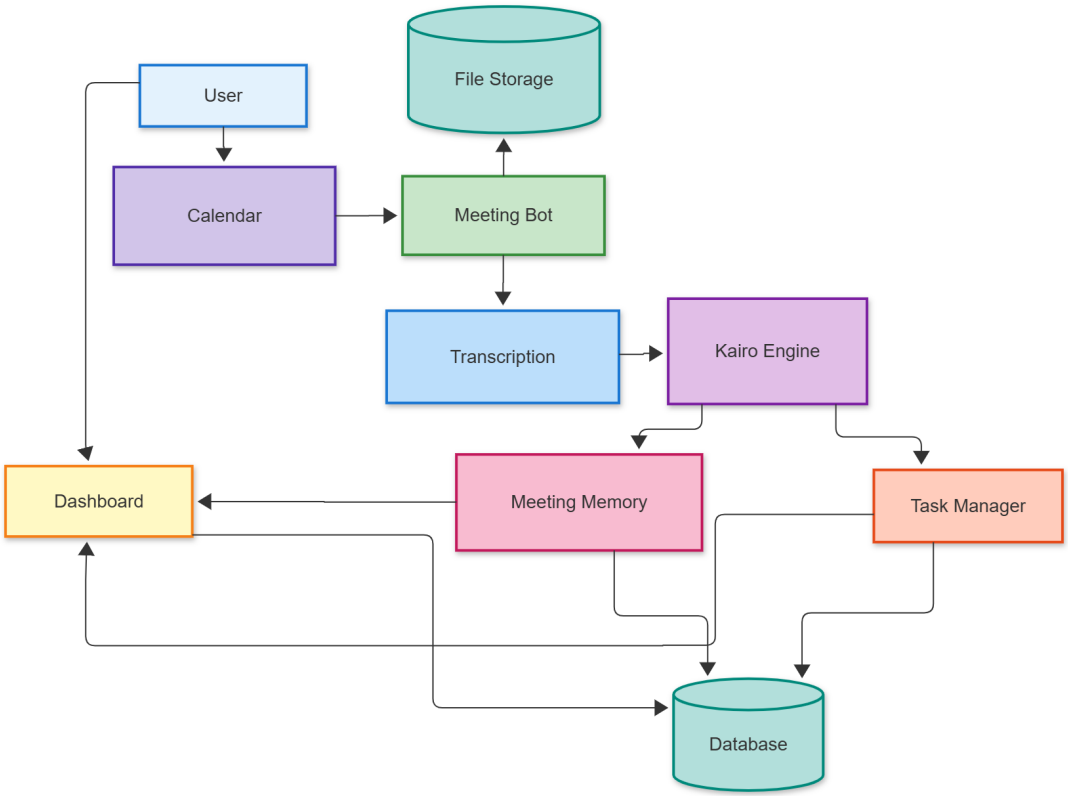

Figure 3.1: Simple System Box and Line Diagram

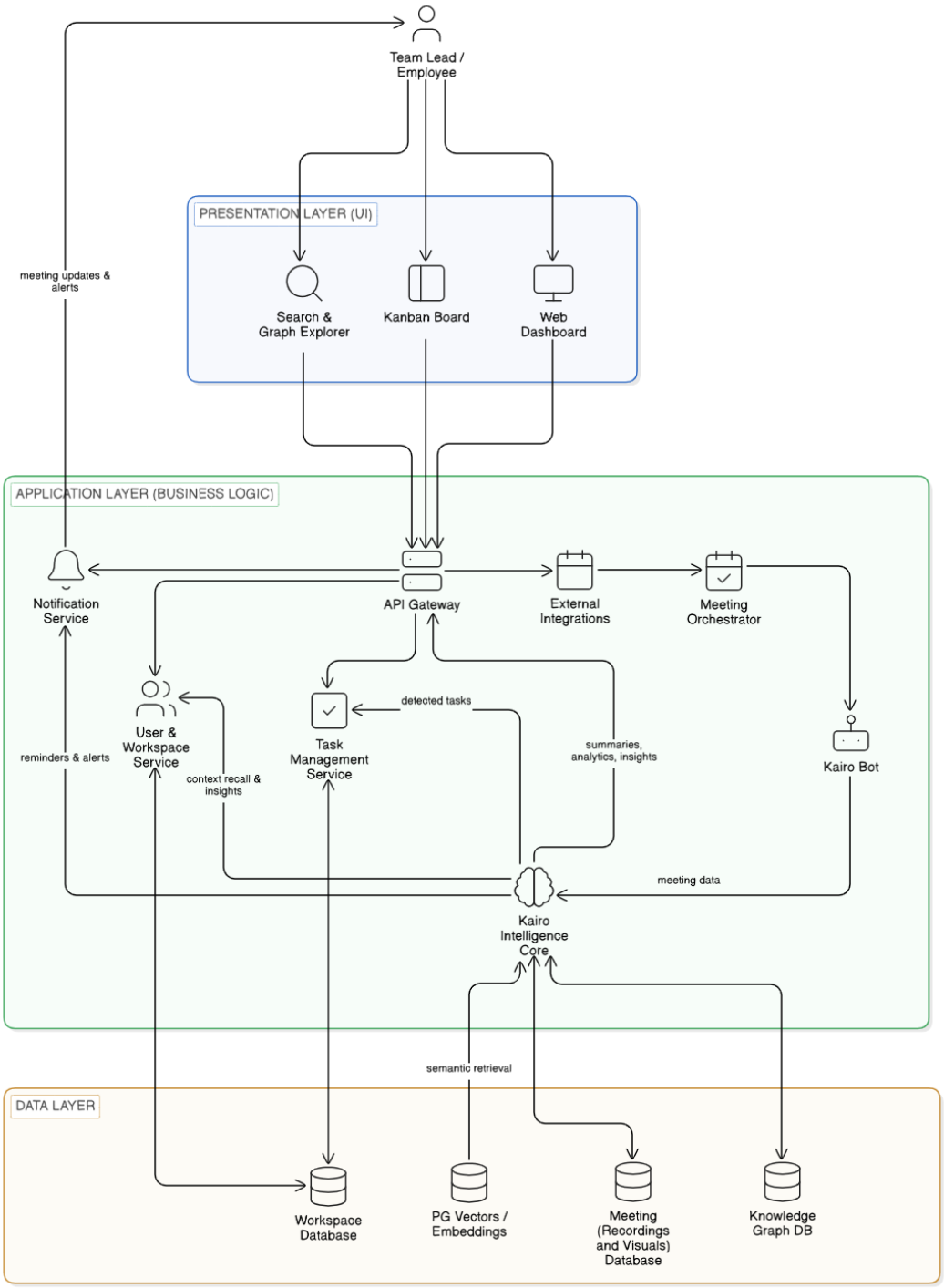

Figure 3.2: 3-Tier System Architectural Design Diagram

3.2 Data Design

The data design of **Kairo** defines how meeting-related information is structured, stored, and processed to support its intelligent collaboration features. The system transforms inputs such as meetings, transcripts, action items, and knowledge graphs into structured relational data, ensuring scalability, integrity, and efficient retrieval.

Kairo employs a unified storage architecture combining **PostgreSQL** for relational and vector data, and **Supabase Storage** for media files (audio, video, images). By leveraging **pgvector**, Kairo integrates semantic search directly within PostgreSQL, eliminating the need for a separate vector database.

3.2.1 Data Storage Architecture

3.2.1.1 Storage Technologies

- **PostgreSQL + pgvector:** Stores all structured relational data including users, meetings, transcripts, action items, and relationships between entities. Additionally, pgvector enables storage and indexing of embedding vectors for semantic similarity search, ensuring tight integration between structured and unstructured data while maintaining ACID compliance.
- **Supabase Storage:** Manages large binary objects (BLOBs) such as meeting recordings (audio/video), screen captures, and profile images. Provides scalable cloud storage with built-in CDN capabilities.

3.2.1.2 Data Organization Layers

Data is organized into modular layers aligned with system functionality:

- **User & Workspace Layer:** Manages users, roles, permissions, and collaborative workspaces.
- **Meeting Layer:** Stores meeting metadata, participant details, and calendar integration tokens.
- **Processing Layer:** Holds transcripts, summaries, action items, tasks, and privacy settings.
- **Knowledge Layer:** Captures semantic knowledge graphs and embedding vectors for cross-meeting insights and contextual search.

- **Media Storage Layer:** References and manages links to audio/video files and visual assets stored in Supabase.

All relational entities are normalized to Third Normal Form (3NF) to reduce redundancy and enforce data integrity through foreign key constraints.

3.2.2 Major Data Entities

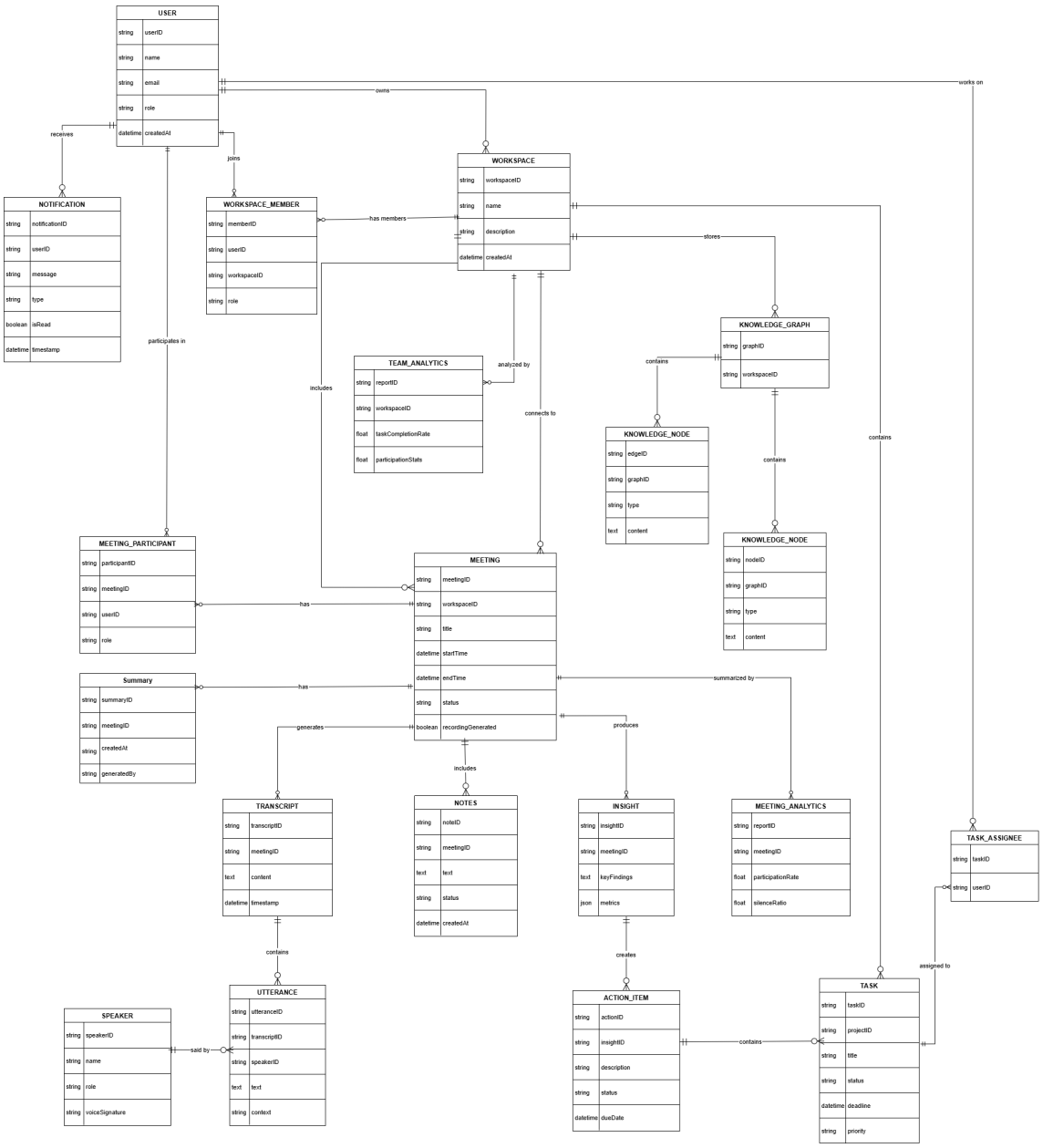

Figure 3.3: Entity-Relationship Diagram of Kairo's Data Model

Table 3.1: Key Entities and Their Relationships

Entity	**Purpose**	**Main Relationships**
`users`	Stores user information, roles, and permissions.	Linked to `workspaces`, `meeting_participants`, and `tasks`.
`workspaces`	Defines collaborative environments for teams.	One-to-many with `users`, `meetings`, and `kanban_boards`.
`calendar_integrations`	Stores calendar OAuth tokens and sync settings.	One-to-many with `meetings`.
`meetings`	Core meeting records with metadata, including references to media files in Supabase.	Linked to `transcripts`, `summaries`, `participants`, and `action_items`.
`meeting_participants`	Tracks participant attendance and roles.	Many-to-one with `meetings` and `users`.
`transcripts`	Stores meeting transcribed text with timestamps.	One-to-one with `meetings`.
`summaries`	Contains AI-generated summaries.	One-to-one with `meetings`.
`action_items`	Key tasks identified during meetings.	One-to-many with `tasks`.
`tasks`	Detailed actionable subtasks with assignments.	Many-to-one with `action_items`.
`kanban_boards`	Visual task-tracking boards.	One-to-one with `workspaces`.
`knowledge_graphs`	Stores semantic linkages and insights.	One-to-one with `workspaces`.
`privacy_controls`	Controls access to sensitive data.	One-to-one with `meetings`.
`media_files`	References to audio/video files stored in Supabase.	Many-to-one with `meetings`.

3.2.3 Data Transformation and Processing Pipeline

The system processes data through the following stages:

1. **Ingestion:** Calendar events and participant details are imported via OAuth integrations (Google Calendar, Outlook).
2. **Capture:** Kairo Bot joins meetings to record audio/video streams and capture screen visuals. Raw media files are uploaded to Supabase Storage with secure access URLs stored in PostgreSQL.
3. **Transcription:** Audio streams are processed through speech-to-text engines, producing timestamped transcripts stored in the `transcripts` table.
4. **Natural Language Processing:** Transcripts are analyzed using NLP pipelines to generate summaries, extract action items, and identify key topics. Results are stored in respective tables (`summaries`, `action_items`).
5. **Semantic Indexing:** Text embeddings are generated using transformer models and stored directly in PostgreSQL using pgvector columns. Vector indexing (e.g., IVF-Flat) enables efficient similarity search across meetings.
6. **Knowledge Graph Construction:** Entities, relationships, and insights are extracted from processed data and stored in `knowledge_graphs`, along with their embeddings, to enable cross-meeting contextual queries.
7. **Retrieval & Access:** Users query historical data through web/mobile interfaces using keyword search, semantic search (via pgvector similarity queries), and structured filters (PostgreSQL queries).

3.2.4 Data Storage Items

- **PostgreSQL Databases:**
  - `kairo_production`: Main production database containing relational tables and embedding vectors
  - `kairo_analytics`: Separate database for aggregated metrics and reporting
- **Supabase Storage Buckets:**
  - `meeting-recordings`: Audio/video files from recorded meetings
  - `screen-captures`: Visual assets and screenshots
  - `user-assets`: Profile images and workspace logos

- **pgvector Embeddings (within PostgreSQL):**
  - `transcript_embeddings`: Vector representations of meeting transcripts stored as pgvector columns
  - `knowledge_embeddings`: Embeddings of knowledge graph entities for semantic linking and retrieval

3.2.5 Summary

Kairo's data architecture combines the reliability of relational databases, the scalability of cloud object storage, and the intelligence of integrated semantic vector search. By leveraging PostgreSQL with pgvector, the system unifies structured data and embeddings within a single database, simplifying architecture while improving performance and consistency. This design ensures that meeting data is captured, structured, and connected across its lifecycle — from scheduling and transcription to action extraction and contextual retrieval — enabling intelligent meeting analytics and efficient cross-meeting knowledge discovery.

3.3 Domain Model

The domain model represents the core structure of the system, identifying its main entities, their attributes, and relationships. It provides a conceptual view of how data and operations interact within the system, serving as a bridge between real-world requirements and the system's design.

In this project, the domain model captures various interconnected modules such as meetings, tasks, analytical reports, users, and insights. Each class (or entity) encapsulates attributes and relationships necessary for efficient project and meeting management. For instance:

Users represent individuals interacting with the system, such as employees, managers, or clients.

Meetings store meeting details like date, time, agenda, and participants.

Tasks are linked to meetings and action items, enabling project tracking.

Analytical Reports and Insights provide summaries and evaluations based on meetings and task performance.

Knowledge graphs and transcripts help maintain contextual understanding and decision-making records.

These entities work together to facilitate seamless collaboration, knowledge sharing, and task execution within the organization.

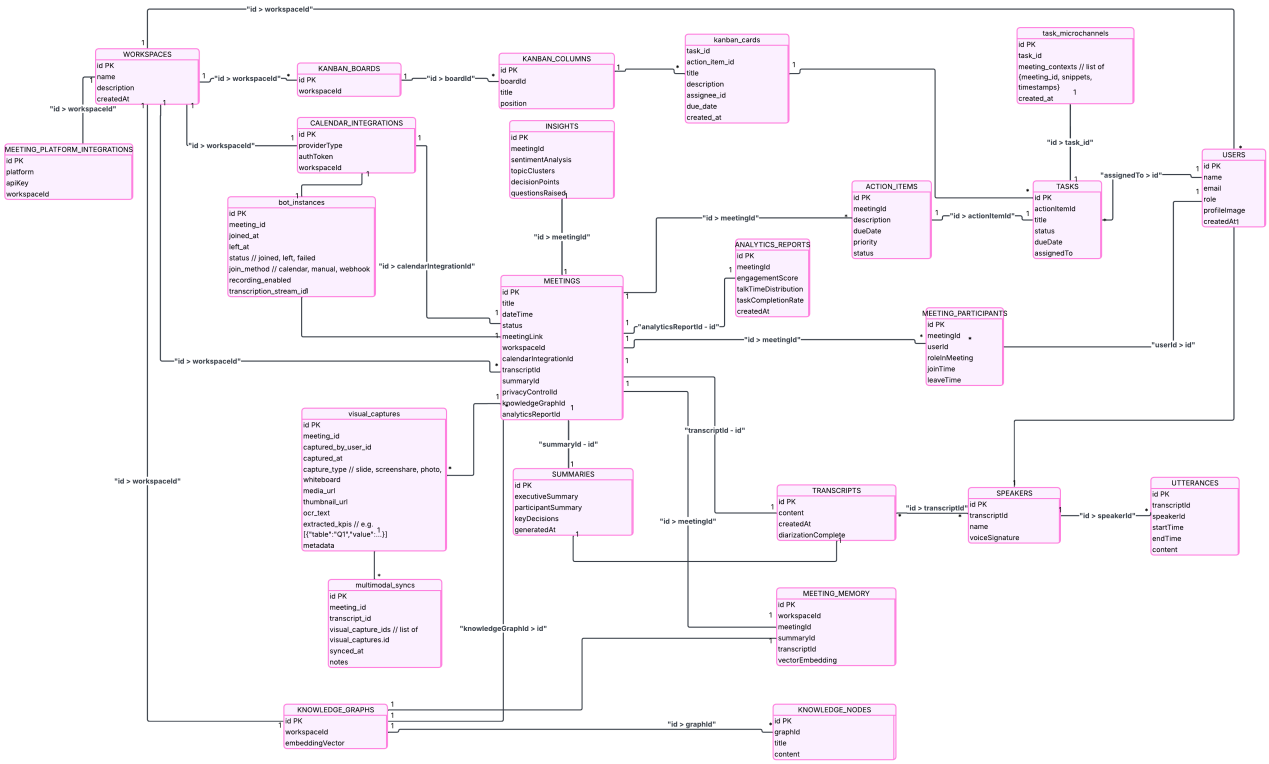

Figure 3.4: Domain Model Diagram of the System

3.4 Design Models

This section presents the core design models developed for the Kairo system using an object-oriented development approach.

3.4.1 Class Diagram

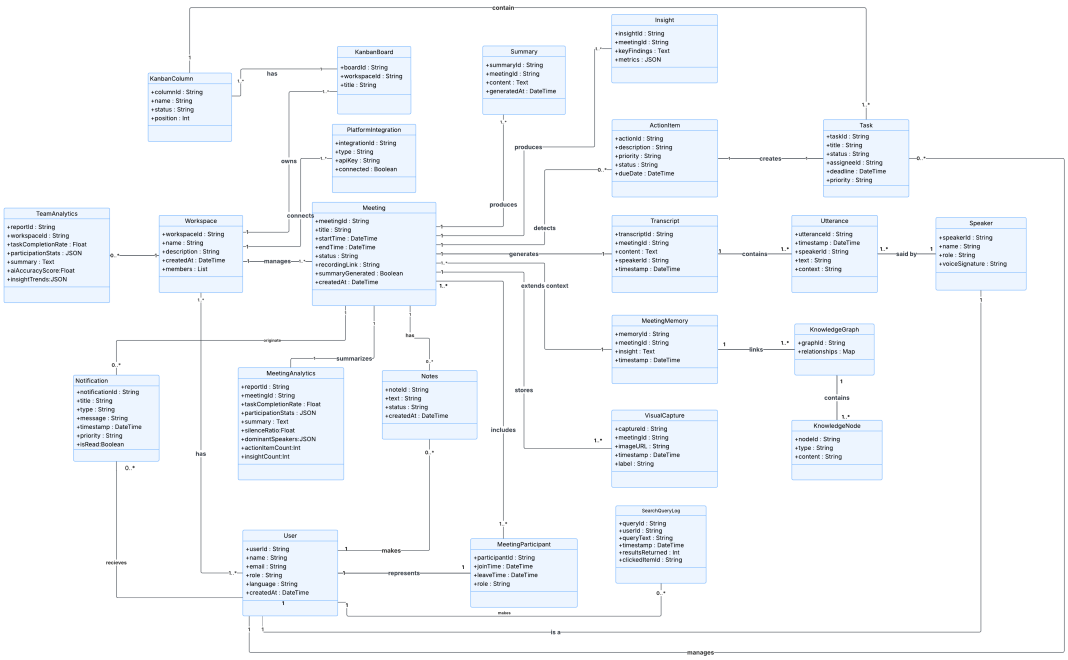

Figure 3.5: Class diagram

3.4.2 Activity Diagram

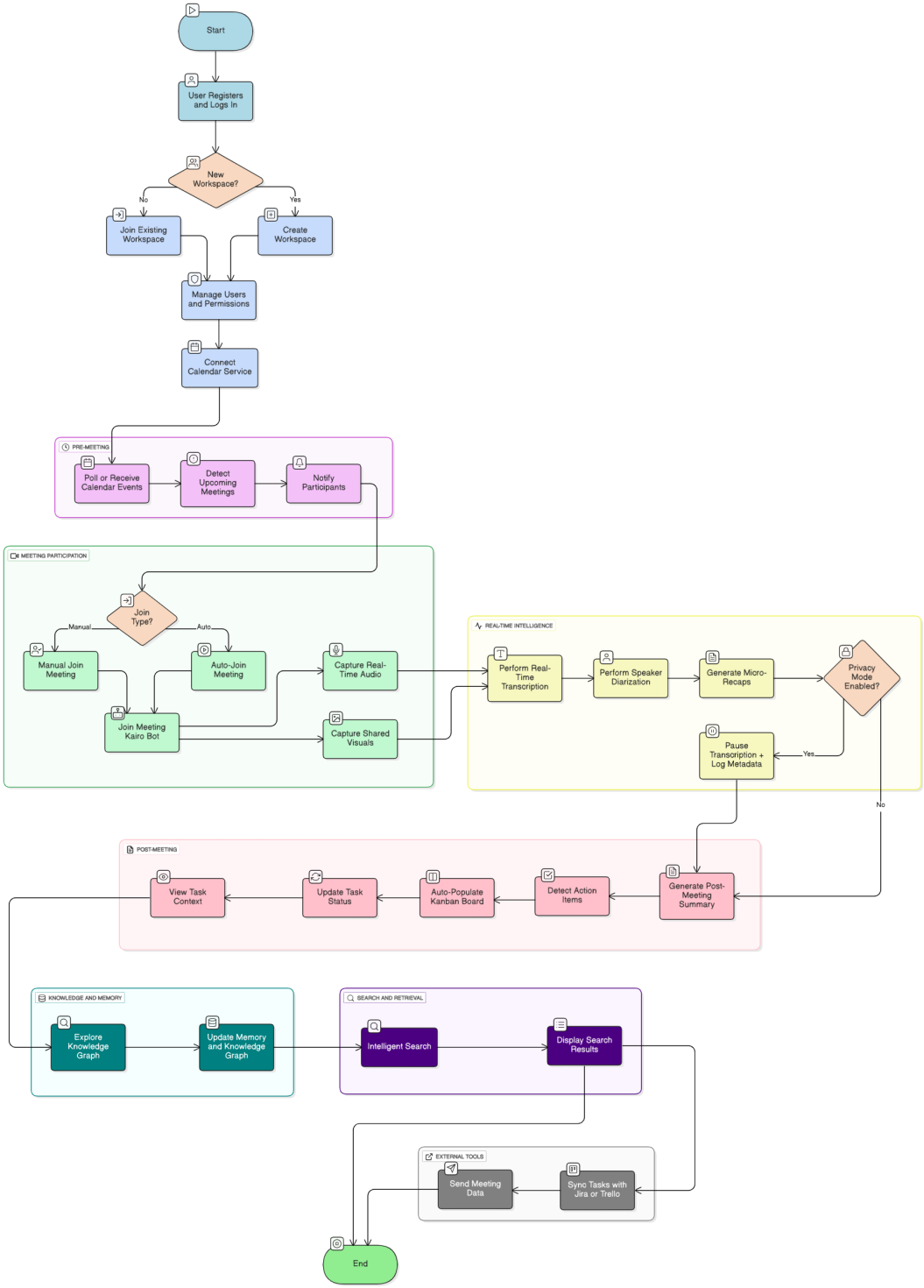

Figure 3.6: Activity Diagram of Kairo showing the meeting lifecycle.

3.4.3 Class-Level Sequence Diagram

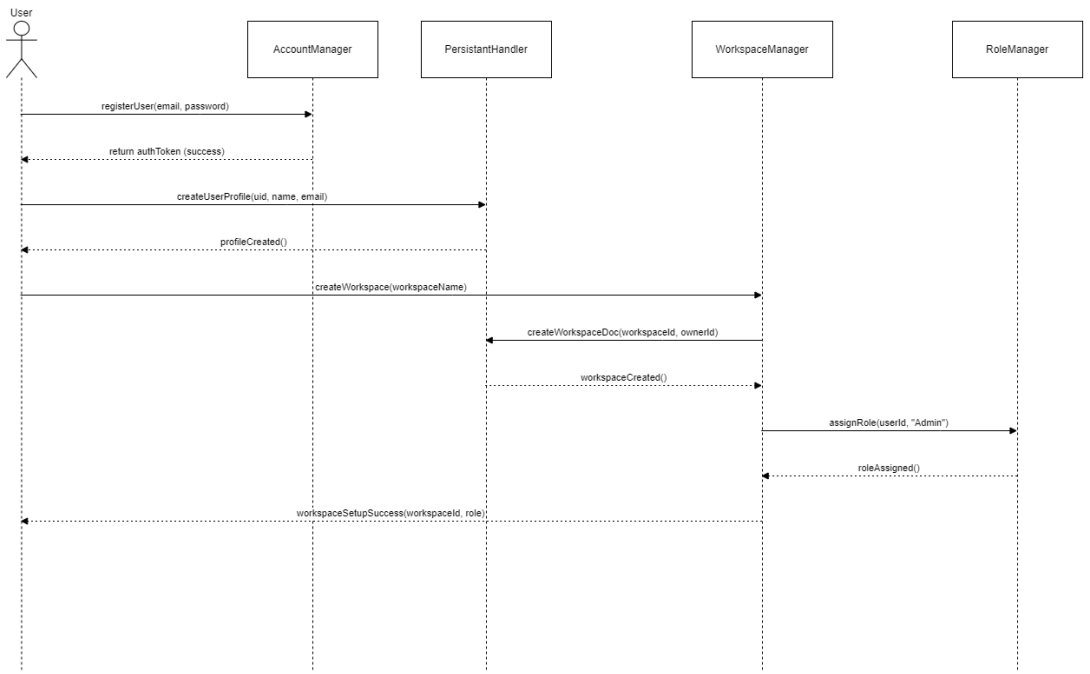

Figure 3.7: Class-level Sequence Diagram for the registration module, highlighting the interactions between user, authentication service, and workspace components.

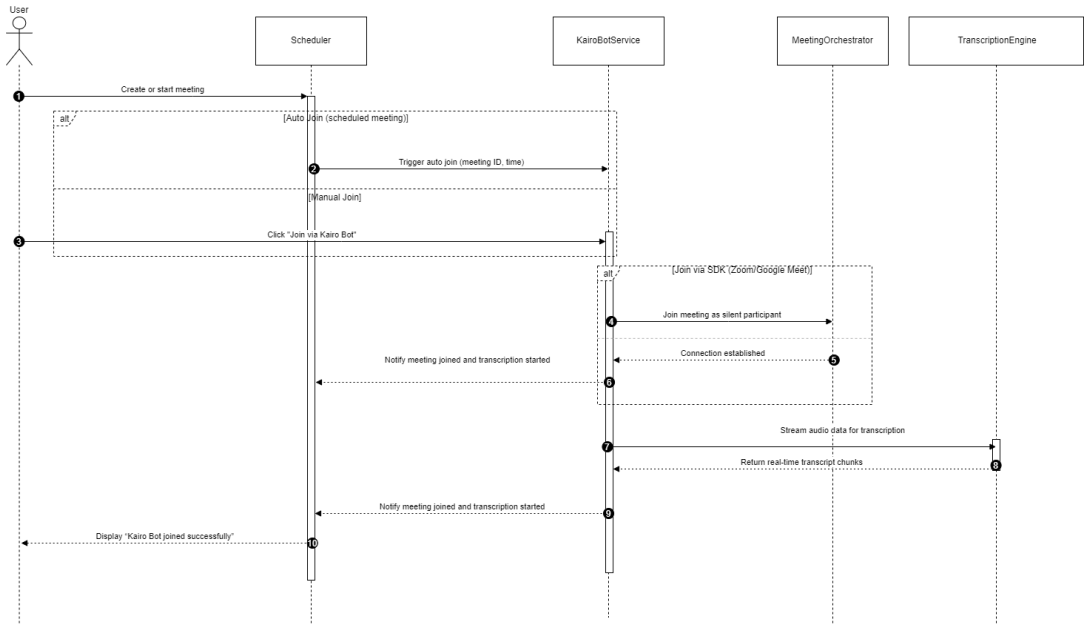

Figure 3.8: Class-level Sequence Diagram illustrating both automatic and manual bot joining processes, followed by transcription initiation.

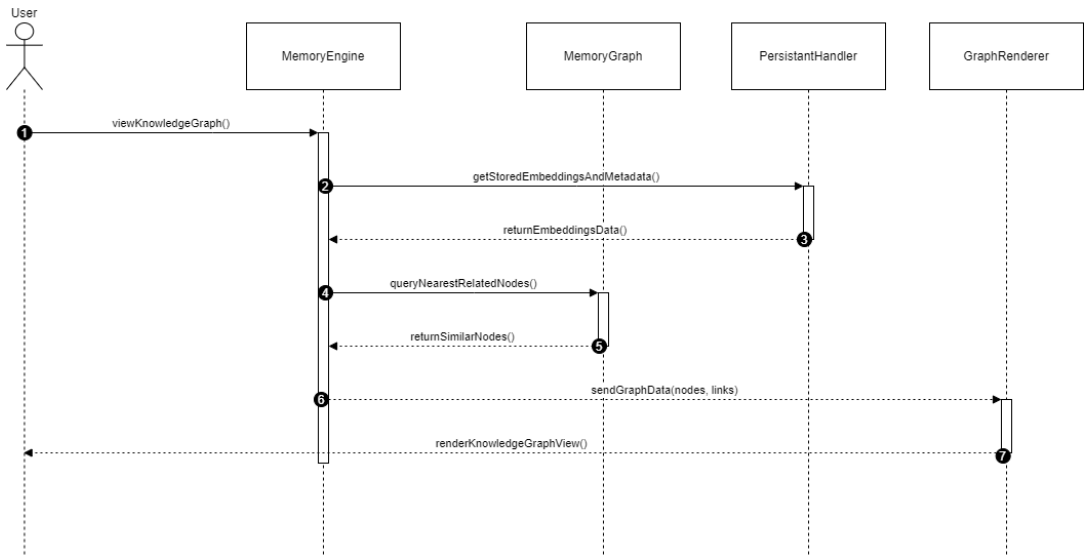

Figure 3.9: Class-level Sequence Diagram for visualizing the Knowledge Graph.

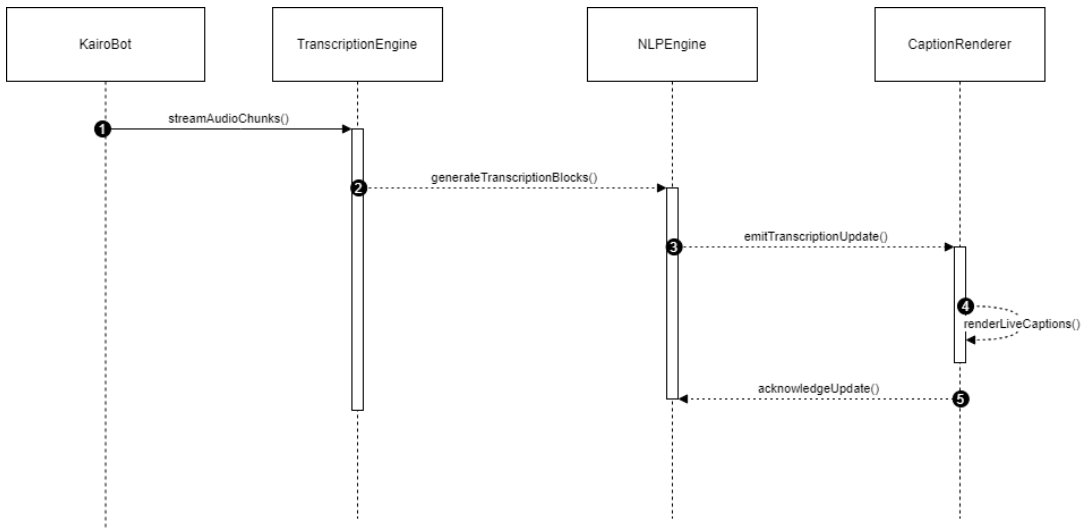

Figure 3.10: Class-Level Sequence Diagram of Live Transcription

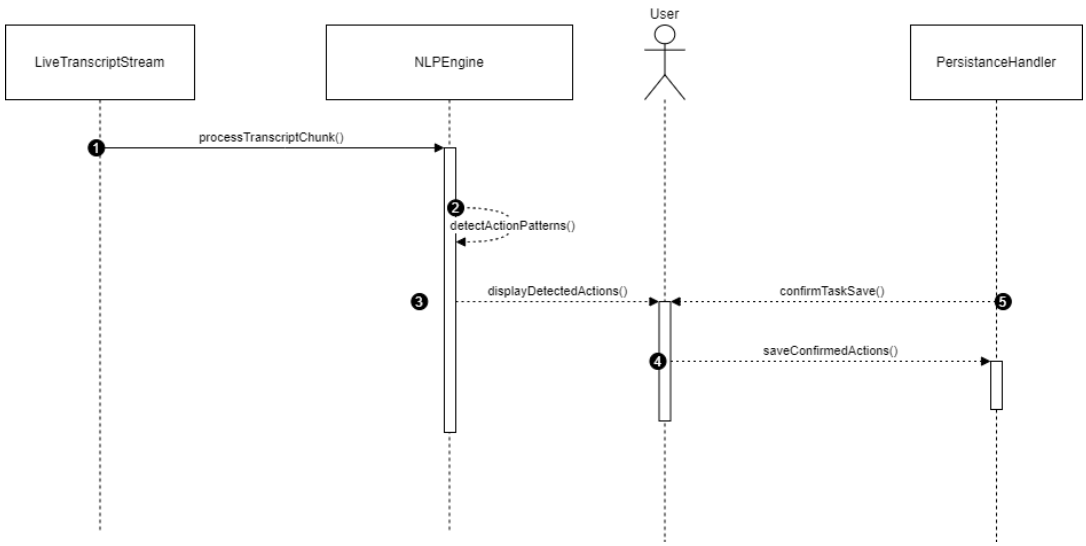

Figure 3.11: Class-level Sequence Diagram illustrating action item detection during meetings.

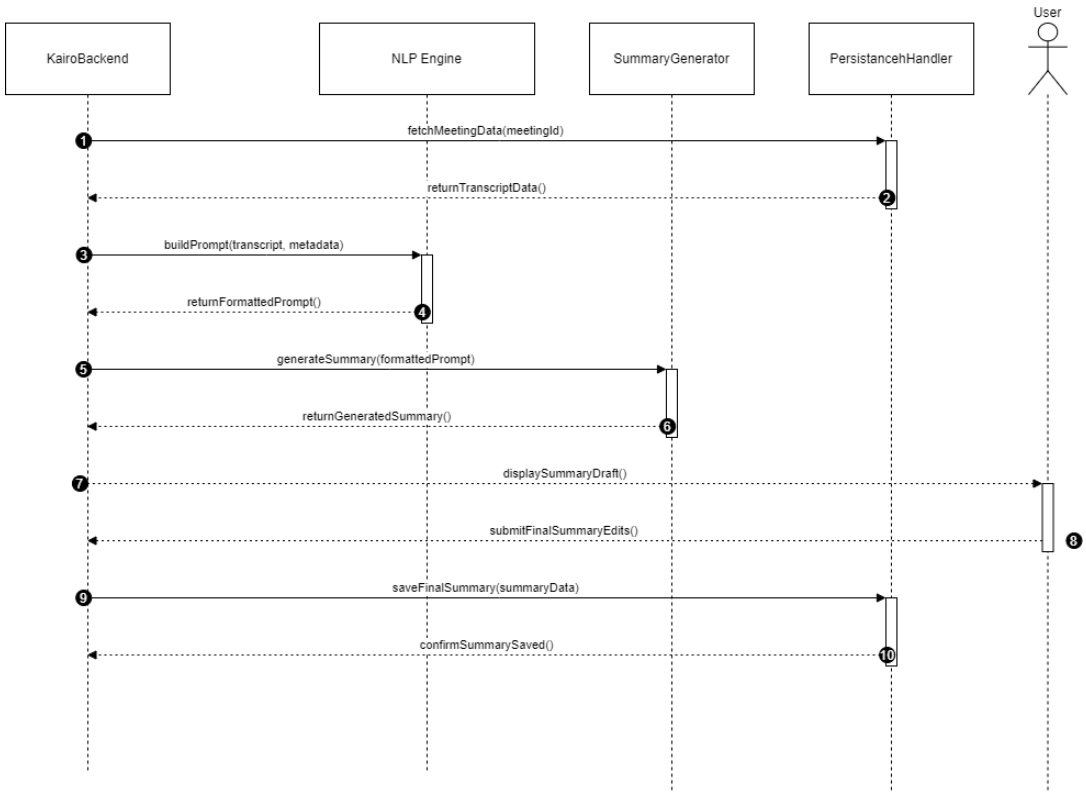

Figure 3.12: Class-level Sequence Diagram for meeting summary generation.

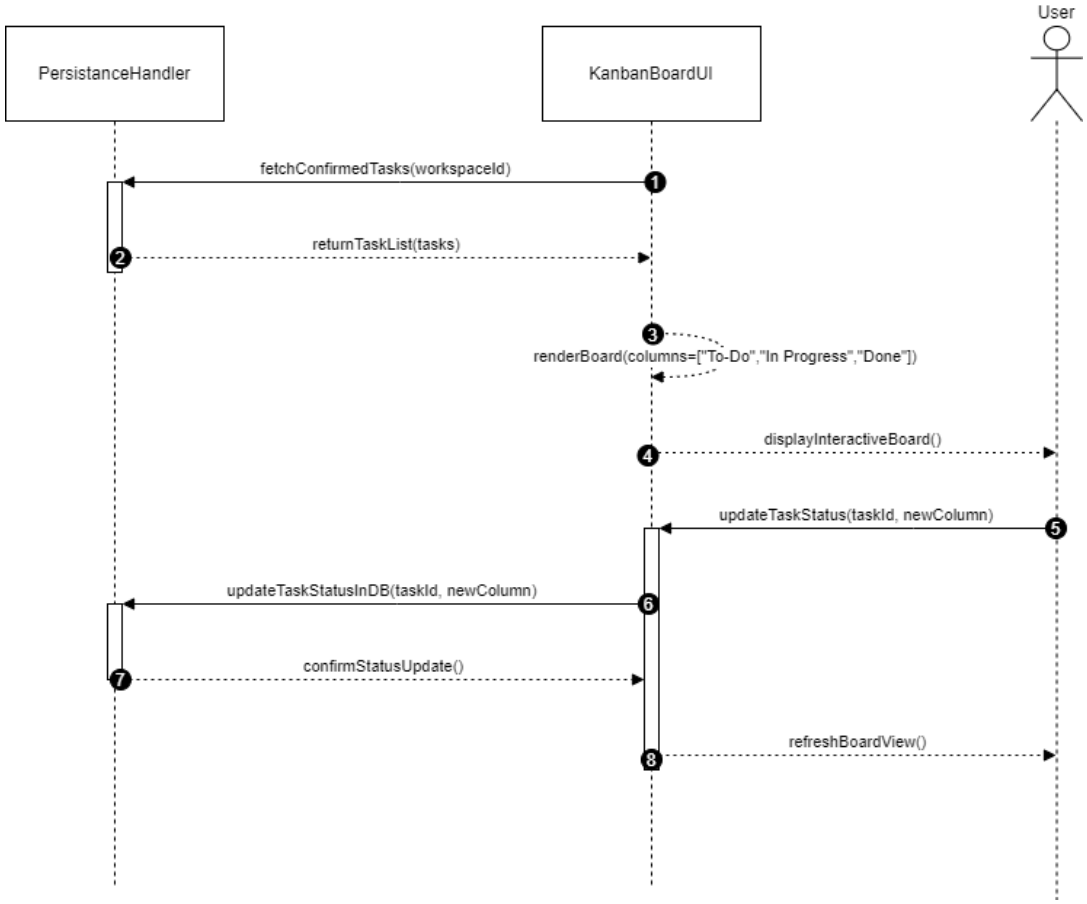

Figure 3.13: Class-level Sequence Diagram depicting Kanban generation within Kairo's task management module.

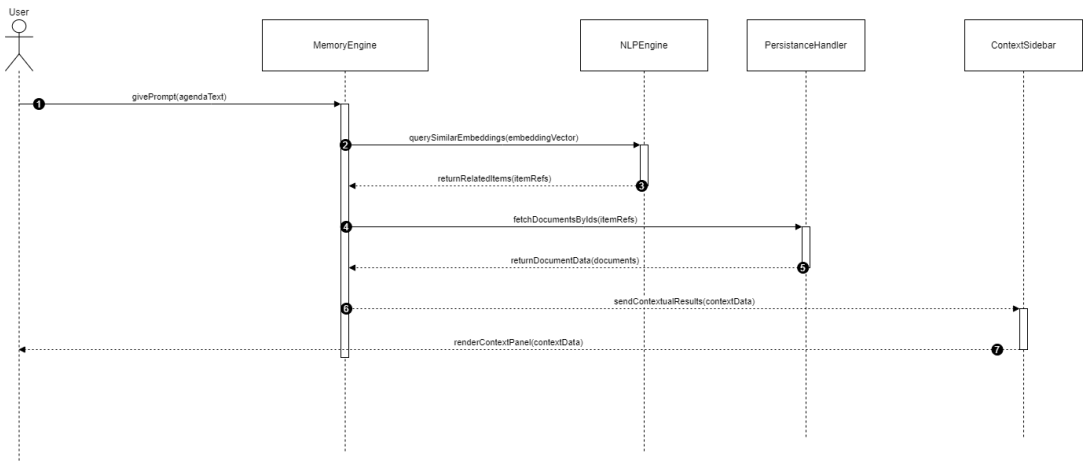

Figure 3.14: Class-level Sequence Diagram depicting the retrieval process from Kairo's Meeting Memory using conversational queries and semantic vector search.

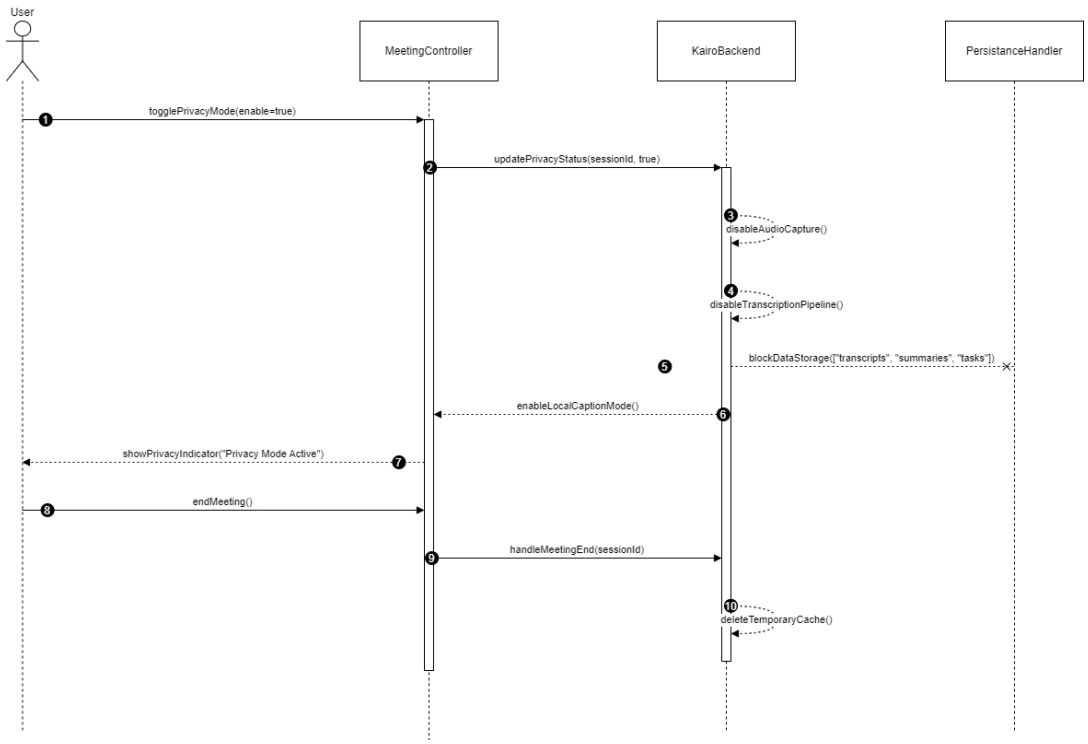

Figure 3.15: Class-level Sequence Diagram illustrating the activation and handling of Privacy Mode in Kairo.

3.4.4 State-Transition Diagram

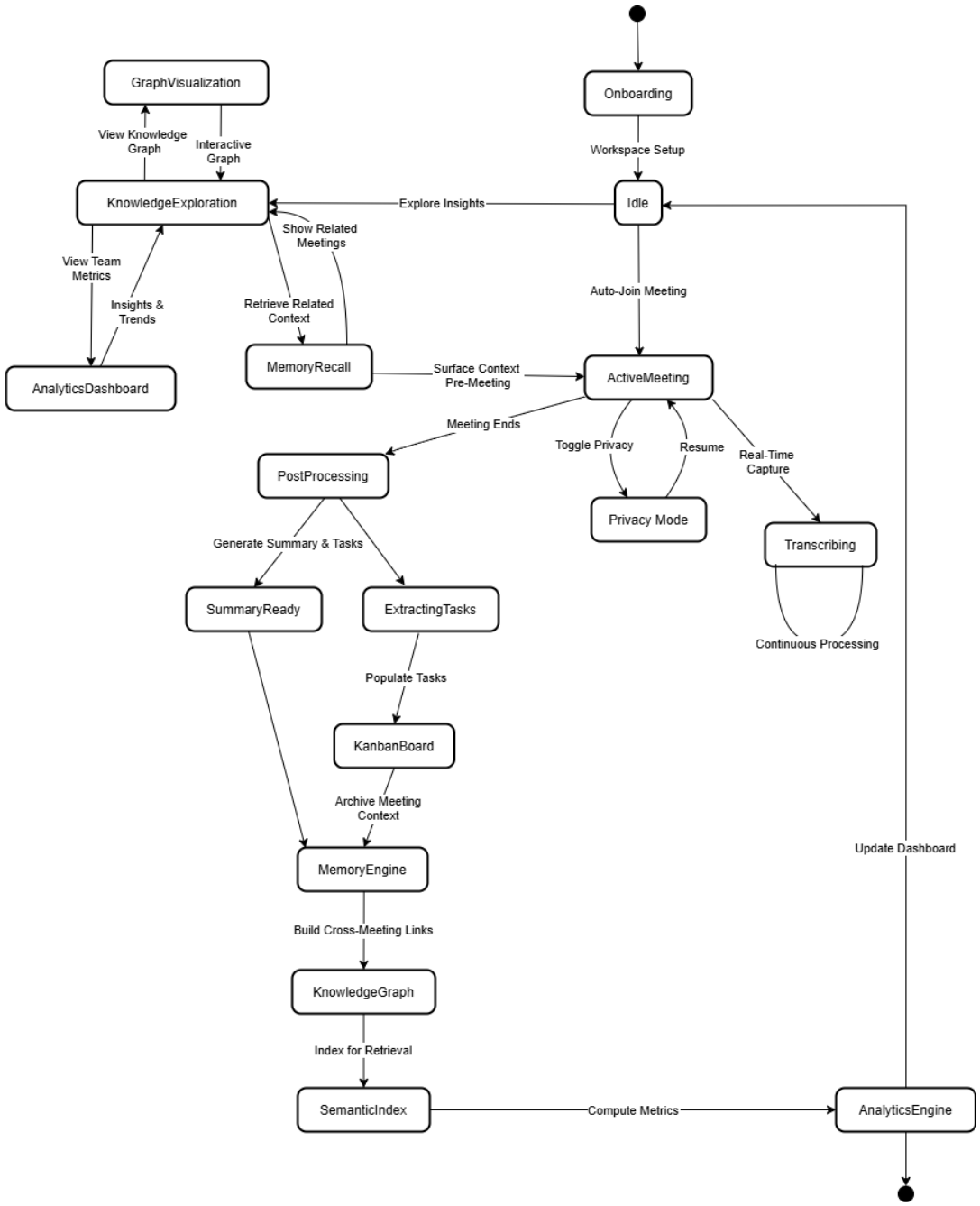

Figure 3.16: State-Transition Diagram of Kairo showing the end-to-end meeting lifecycle.

Chapter 4

Implementation and Testing

This chapter outlines the development and testing of the Kairo intelligent meeting platform across all four iterations, representing the completion of the entire FYP. Building upon the foundation established in Iterations #1 and #2, Iterations #3 and #4 introduce complete meeting capture, contextual task management, knowledge graph visualization, analytics-driven insights, integrated semantic search using pgvector within PostgreSQL, and calendar integration for automated meeting synchronization.

The chapter details the implementation of key components such as synchronized transcript and speaker identification, Meeting Memory Graph construction and visualization, task contextual micro-channels, and analytics modules for extracting metrics like participation patterns and discussion trends. It also highlights the use of pgvector for storing and querying embedding vectors directly within PostgreSQL, enabling efficient semantic retrieval without relying on external vector databases. These features collectively enhance Kairo's ability to transform meetings into structured, actionable, and visually interpretable knowledge.

Additionally, the chapter covers the integration of external APIs and frameworks, including AI/ML services for transcription, natural language processing, and embedding generation, as well as tools for graph visualization and data analytics. Their roles and interactions within the system architecture are clearly defined.

Finally, the chapter presents the unit and integration testing strategies used to ensure system reliability, performance, and accuracy across both traditional and advanced features, ensuring a robust and seamless user experience in preparation for the final system demonstration.

4.1 Algorithm Design

This section presents the core algorithms implemented in Kairo's major modules. These algorithms enable the system to process unstructured meeting data and transform it into actionable intelligence.

No.	Algorithm	Reference
1	User Sign Up and Login	Algorithm 1
2	Workspace Creation and Member Management	Algorithm 2
3	Meeting Scheduling	Algorithm 3
4	Automated Bot Meeting Join via Puppeteer	Algorithm 4
5	Real-Time Meeting Transcription Processing	Algorithm 5
6	Speaker Diarization Using Audio Profiles	Algorithm 6
7	Action Item Detection and Extraction	Algorithm 7
8	Intelligent Meeting Summary Generation	Algorithm 8
9	Whisper Mode Micro-Recap Generation	Algorithm 9
10	Post-Meeting Processing Pipeline	Algorithm 10
11	Meeting Memory Graph Generation	Algorithm 11
12	Workspace Semantic Search	Algorithm 12
13	Privacy Mode: Transcription Suppression and Insight Filtering	Algorithm 13
14	Calendar Integration and Event Synchronization	Algorithm 14

Table 4.1: Summary of Algorithms Implemented in Kairo

4.1.1 User Sign Up and Login Algorithm

The user authentication algorithm handles user registration and login, providing secure access to the Kairo platform.

Algorithm 1 User Sign Up and Login

Require: User credentials: email *email*, password *password*

Ensure: Authenticated user session or new user account

```
if operation is sign up then
    Validate email format and uniqueness
    Validate password strength (minimum 8 characters, complexity requirements)
    if validation passes then
        Generate password hash: hash ← HashPassword(password)
        Create user record: user ← {email, password_hash : hash, created_at}
        Store user in users table
        Generate session token token
        Create session: {user_id, token, expires_at}
        return {success, token, user_id}
    else
        return {error, validation_message}
    end if
else
    Retrieve user record by email email
    if user exists then
        Verify password: valid ← VerifyPassword(password, user.password_hash)
        if valid then
            Generate session token token
            Create session: {user_id : user.id, token, expires_at}
            return {success, token, user_id : user.id}
        else
            return {error, "Invalidcredentials"}
        end if
    else
        return {error, "Usernotfound"}
    end if
end if
```

4.1.2 Workspace Creation and Member Management Algorithm

This algorithm handles workspace creation and manages member access with role-based access control.

Algorithm 2 Workspace Creation and Member Management

Require: User ID $user_{id}$, workspace name *name*

Ensure: Workspace created with owner as member

1: Validate workspace name (non-empty, unique for user)
2: Generate unique workspace ID w_{id}
3: Create workspace record: $W \leftarrow \{w_{id}, name, owner : user_{id}, created_at\}$
4: Store W in workspaces table
5: Add owner to workspace_members with role "admin"
6: Initialize empty kanban board for workspace
7: Initialize empty knowledge graph for workspace
8: Create default workspace settings: $\{privacy_mode : false, auto_join_bot : true\}$
9: Log workspace creation event
10: **return** w_{id}

4.1.3 Meeting Scheduling Algorithm

The meeting scheduling algorithm creates meetings and configures bot auto-join.

Algorithm 3 Meeting Scheduling

Require: Workspace ID w_{id}, meeting details $details$, participant IDs $participants[]$

Ensure: Meeting scheduled with bot auto-join configured

1: Validate meeting details: $\{title, start_time, duration, meeting_url\}$
2: Check participant permissions in workspace
3: Generate unique meeting ID m_{id}
4: Create meeting record: $M \leftarrow \{m_{id}, w_{id}, title, start_time, duration, status : \text{"}scheduled\text{"}\}$
5: Store M in meetings table
6: **for** each participant in $participants[]$ **do**
7: Add to meeting_participants: $\{m_{id}, user_id : participant, status : \text{"}invited\text{"}\}$
8: **end for**
9: Calculate bot join time: $join_time \leftarrow start_time - 30$ seconds
10: Schedule bot join task: $\{m_{id}, join_time, meeting_url\}$
11: Add to job queue for automated execution
12: Send in-app notifications to all participants
13: **return** m_{id}

4.1.4 Bot Meeting Joining Algorithm

The bot meeting joining algorithm uses browser automation (Puppeteer) to programmatically join online meetings via Google Meet or Zoom and initiate recording. The bot always joins muted with camera off.

Algorithm 4 Automated Bot Meeting Join via Puppeteer

Require: Meeting URL url, meeting ID m_{id}, bot credentials $creds$
Ensure: Bot successfully joined and recording initiated

```
Launch headless browser instance using Puppeteer
page ← browser.newPage()
Navigate to meeting URL: page.goto(url)
Wait for page load and meeting interface to render
if authentication required then
    Enter bot credentials creds in login form
    Submit authentication
    Wait for redirect to meeting room
end if
Locate and click "Join Meeting" button
Wait for audio/video permissions dialog
Accept microphone permission
Disable camera (set camera off)
Mute microphone (set muted)
if waiting room enabled then
    Wait for host admission
    Monitor admission status every 2 seconds
end if
Verify bot has joined successfully (check DOM for participant list)
Initialize audio capture from meeting stream
Update meeting status in database: status ← "bot_joined"
Start recording and transcription pipeline
Send webhook notification to application backend
return success status
```

4.1.5 Real-Time Transcription Processing Algorithm

The real-time transcription processing algorithm handles continuous audio streams from active meetings, converting speech to text with speaker diarization and timestamp alignment.

Algorithm 5 Real-Time Meeting Transcription Processing

Require: Meeting audio stream A, meeting ID m_{id}
Ensure: Timestamped transcript with speaker labels stored in database

```
Initialize audio buffer B ← ∅
Initialize transcript array T ← []
Connect to meeting audio stream A
while meeting is active do
    Capture audio chunk c from stream A
    Append c to buffer B
    if |B| ≥ threshold_size then
        text ← SpeechToText(B)
        speaker ← IdentifySpeaker(B)
        timestamp ← GetCurrentTimestamp()
        Create transcript segment s ← {text, speaker, timestamp}
        Append s to T
        Store s in database with m_id
        EmitRealTimeUpdate(s) {Send to connected clients}
        Clear buffer B ← ∅
    end if
end while
Finalize transcript T in database
return T
```

4.1.6 Speaker Diarization Algorithm

The speaker diarization algorithm identifies different speakers in a meeting by analyzing audio characteristics and user audio profiles.

Algorithm 6 Speaker Diarization Using Audio Profiles

Require: Audio chunks $chunks[]$, participant profiles $profiles[]$, meeting ID m_{id}
Ensure: Speaker-labeled transcript segments

```
Initialize speaker segments list segments ← []
Load voice embeddings for all participants from profiles[]
for each audio chunk c in chunks[] do
    Extract audio features: features ← ExtractMFCC(c)
    Generate voice embedding: embedding ← EmbedVoice(features)
    Initialize similarity scores scores ← {}
    for each profile p in profiles[] do
        similarity ← CosineSimilarity(embedding, p.voice_embedding)
        scores[p.user_id] ← similarity
    end for
    speaker ← argmax(scores)
    if max(scores) < 0.6 then
        speaker ← "Unknown_Speaker"
        Create new temporary profile for unknown speaker
    end if
    Transcribe audio chunk: text ← SpeechToText(c)
    timestamp ← GetTimestamp(c)
    Create segment: seg ← {speaker, text, timestamp, confidence : max(scores)}
    Append seg to segments
    if speaker is new then
        Update user audio profile with new embedding data
        Store updated profile in database
    end if
end for
Apply speaker consistency check (merge adjacent same-speaker segments)
Store all segments in transcripts table with m_id
return segments
```

4.1.7 Action Item Extraction Algorithm

The action item extraction algorithm analyzes meeting transcripts using natural language processing to identify actionable tasks, decisions, and commitments made during discussions.

Algorithm 7 Action Item Detection and Extraction

Require: Transcript T, meeting context C
Ensure: List of extracted action items AI

```
Initialize action items list AI ← []
Split transcript T into sentences S = {s_1, s_2, ..., s_n}
for each sentence s_i in S do
    features ← ExtractNLPFeatures(s_i)
    if ContainsActionKeywords(features) then
        context_window ← GetSurroundingSentences(s_i, S, window_size=3)
        embedding ← GenerateEmbedding(context_window)
        classification ← ClassifyAsActionItem(embedding)
        if classification.confidence > 0.6 then
            task ← ExtractTaskDescription(s_i)
            assignee ← IdentifyAssignee(context_window, participants)
            deadline ← ExtractDeadline(context_window)
            priority ← EstimatePriority(s_i, C)
            Create action item ai ← {task, assignee, deadline, priority}
            Append ai to AI
        end if
    end if
end for
AI ← DeduplicateActionItems(AI)
AI ← RankByPriority(AI)
return AI
```

4.1.8 Meeting Summary Generation Algorithm

The summary generation algorithm creates concise, structured summaries by identifying key discussion points and decisions.

Algorithm 8 Intelligent Meeting Summary Generation

Require: Transcript T, meeting metadata M, action items AI
Ensure: Structured summary S

```
segments ← SegmentByTopic(T)
Initialize summary components S ← {}
S.title ← M.title
S.participants ← M.participants
S.duration ← M.duration
key_points ← []
for each segment seg in segments do
    topic ← ExtractTopic(seg)
    importance_score ← CalculateImportance(seg, T)
    if importance_score > threshold then
        summary_text ← ExtractiveSummarization(seg)
        Append {topic, summary_text, importance_score} to key_points
    end if
end for
S.key_points ← SortByImportance(key_points)
S.decisions ← ExtractDecisions(T)
S.action_items ← AI
S.next_steps ← SynthesizeNextSteps(AI, S.decisions)
S.summary_text ← GenerateAbstractiveSummary(S)
return S
```

4.1.9 Whisper Mode Micro-Recap Algorithm

The Whisper Mode algorithm generates concise micro-recaps during live meetings by periodically analysing recent transcript segments and broadcasting summaries to active participants via WebSocket.

Algorithm 9 Whisper Mode Micro-Recap Generation

Require: Active meeting M, transcript buffer B, recap interval Δt, LLM service $\mathcal{L}$
Ensure: Timestamped micro-recap R broadcast to connected clients

```
Initialize last recap timestamp t_last ← M.startTime
Initialize recap history H ← []
while M.status = in-progress do
    t_now ← CurrentTime()
    if (t_now − t_last) ≥ Δt or UserRequestedRecap(M) then
        segments ← FetchRecentSegments(B, t_last, t_now)
        if |segments| = 0 then
            continue
        end if
        context ← BuildPromptContext(segments, H)
        R_raw ← L.Generate(context, max_sentences=3)
        R ← {text: R_raw, timestamp: t_now, spanStart: t_last, spanEnd: t_now}
        Append R to H
        BroadcastViaWebSocket(M.connectedClients, R)
        PersistRecap(M.id, R)
        t_last ← t_now
    end if
    if NewParticipantJoined(M) then
        R_join ← GenerateCatchUpRecap(B, M.startTime, t_now, L)
        BroadcastToParticipant(M.newParticipant, R_join)
    end if
    Sleep(δ) {Polling interval, e.g. 10 seconds}
end while
```

4.1.10 Post-Meeting Processing Pipeline

The post-meeting pipeline is an orchestrated sequence of asynchronous stages that executes automatically upon meeting termination, handling resource cleanup, transcript refinement, AI insight generation, human approval gating, and knowledge base updates.

Algorithm 10 Post-Meeting Processing Pipeline

Require: Meeting record M, audio buffer $\mathcal{A}$, raw transcript chunks $\mathcal{T}_{raw}$, workspace W
Ensure: Persisted insights $\mathcal{I}$, updated vector store $\mathcal{V}$, updated knowledge graph $\mathcal{G}$

1: **// — Phase 1: Resource Capture & Finalisation —**
2: $M.status \leftarrow$ `ending`; BroadcastStatus(M, `meeting_ending`)
3: $\mathcal{A}_{file} \leftarrow$ FinaliseAndMergeAudio($\mathcal{A}$, $M.id$)
4: PersistRecording($\mathcal{A}_{file}$, M)
5: **// — Phase 2: Bot Departure & Cleanup —**
6: LeaveMeeting($M.botSession$)
7: CloseBrowserSession($M.botSession$); ReleaseBotLock($M.id$)
8: DeleteTempFiles($M.id$)
9: $M.status \leftarrow$ `processing`; $M.endTime \leftarrow$ CurrentTime()
10: **// — Phase 3: Full Diarized Transcript Generation —**
11: $\mathcal{T}_{merged} \leftarrow$ MergeRawChunks($\mathcal{T}_{raw}$)
12: $\mathcal{D} \leftarrow$ RunDiarization($\mathcal{A}_{file}$)
13: $\mathcal{T}_{final} \leftarrow$ AlignAndResolveSpeakers($\mathcal{T}_{merged}$, $\mathcal{D}$, $W.voiceProfiles$)
14: PersistTranscript($M.id$, $\mathcal{T}_{final}$)
15: BroadcastStatus(M, `transcript_ready`)
16: **// — Phase 4: Parallel AI Agent Execution —**
17: $\mathcal{I} \leftarrow$ RunAgentsInParallel($\mathcal{T}_{final}$, M, W) {Summary, decisions, action items, sentiment, topics, participant analysis}
18: **for** each failed agent result r in $\mathcal{I}$ **do**
19: $\quad r \leftarrow$ RetryOrMarkNull(r, max_retries=2)
20: **end for**
21: PersistInsights($M.id$, $\mathcal{I}$); BroadcastStatus(M, `insights_ready`)

Algorithm 10 Post-Meeting Processing Pipeline (continued)

```
// — Phase 5: Scheduler Approval Gate —
SendApprovalNotification(GetScheduler(M), 𝓘)
response ← AwaitSchedulerResponse(timeout=T_approval)
if response = timeout then
    𝓘 ← AutoApprove(𝓘)
else if response.action = approved then
    𝓘 ← ApplyEdits(𝓘, response.edits)
else if response.action = rejected then
    NotifyWorkspace(W, insights_rejected)
    return {Halt pipeline; no downstream updates}
end if
PersistInsights(M.id, 𝓘)
// — Phase 6: Task Materialisation —
for each action item a in 𝓘.actionItems do
    if a.confidence ≥ θ_confirm then
        CreateAndNotifyTask(a, W, M.id)
    else
        FlagForManualReview(a, M.id)
    end if
end for
// — Phase 7: Vector Knowledge Base Update —
UpdateVectorStore(M.id, 𝓣_final, 𝓘, 𝓥) {Embeds transcript chunks, summary, decisions, and action items into 𝓥}
// — Phase 8: Knowledge Graph Update —
UpdateKnowledgeGraph(M, 𝓘, 𝓣_final, 𝓖, 𝓥) {Delegates to Meeting Memory Graph Generation algorithm}
// — Phase 9: Finalisation —
M.status ← completed; PersistMeetingRecord(M)
BroadcastStatus(M, processing_complete); NotifyWorkspace(W, M)
return 𝓘, 𝓥, 𝓖
```

4.1.11 Meeting Memory Graph Generation Algorithm

The Meeting Memory Graph algorithm constructs a persistent knowledge graph by extracting typed nodes and edges from meeting artefacts, then detecting cross-meeting relationships using semantic similarity and shared entities.

Algorithm 11 Meeting Memory Graph Generation

Require: Meeting record M, approved insights $\mathcal{I}$, transcript $\mathcal{T}$, graph $\mathcal{G}$, vector store $\mathcal{V}$
Ensure: Updated knowledge graph $\mathcal{G}$

```
// — Phase 1: Node & Edge Construction —
n_M ← UpsertNode(G, type=Meeting, M)
for each p in M.attendees do
    UpsertNode(Participant, p); UpsertEdge(n_p → n_M, attended)
end for
for each t in I.topics do
    UpsertNode(Topic, t); UpsertEdge(n_M → n_t, discussed, weight=t.prominence)
end for
for each d in I.decisions do
    UpsertNode(Decision, d); UpsertEdge(n_M → n_d, decided)
end for
for each a in I.actionItems do
    UpsertNode(Task, a); UpsertEdge(n_M → n_a, produced)
    UpsertEdge(n_a → Node(a.assignee), assigned_to)
end for
entities ← ExtractNamedEntities(T)
for each e in entities do
    UpsertNode(e.type, e); UpsertEdge(n_M → n_e, mentions, weight=e.frequency)
end for
// — Phase 2: Cross-Meeting Relationship Detection —
related ← FindSimilarMeetings(Embed(I.summary), V, top_k=10)
for each M' in related do
    if related[M'].score ≥ θ_relate then
        UpsertEdge(n_M ↔ n_M', semantically_related, weight=related[M'].score)
    end if
    if |IntersectParticipants(M, M')| > 0 then
        UpsertEdge(n_M ↔ n_M', shared_participants)
    end if
end for
for each prior task n_a in G do
    if TaskMentionedIn(n_a, T) then
        UpsertEdge(n_M → n_a, followed_up)
    end if
end for
PersistGraphDelta(G)
return G
```

4.1.12 Semantic Search Algorithm

The semantic search algorithm processes a natural language query by generating a vector embedding and performing hybrid retrieval combining pgvector similarity search with keyword filtering, returning ranked and grouped results across all accessible workspace artefacts.

Algorithm 12 Workspace Semantic Search

Require: Query q, workspace W, user U, vector store $\mathcal{V}$, options $\mathcal{O}$
Ensure: Ranked, grouped results $\mathcal{R}$

```
// — Query Preparation —
filters ← ExtractStructuredFilters(q, O) {Dates, participants, content types}
e_q ← GenerateEmbedding(RemoveFilterTokens(q, filters))
scope ← GetAccessibleMeetings(U, W) {Enforce RBAC before any search}
if |scope| = 0 then
    return ∅
end if
// — Hybrid Retrieval —
C_vec ← pgvectorSearch(V, e_q, scope, O.contentTypes, top_k=K)
C_kw ← FullTextSearch(q, scope, filters)
C ← Deduplicate(C_vec ∪ C_kw)
// — Scoring & Fusion —
for each candidate c in C do
    c.score ← α · s_vec(c) + β · s_kw(c) + γ · RecencyBoost(c)
end for
C ← TopK(SortDescending(C, key=score), K_final)
// — Enrichment & Response —
for each c in C do
    c.snippet ← ExtractHighlightSnippet(c, q)
    c.breadcrumb ← BuildBreadcrumb(c)
end for
R ← GroupByContentType(C)
return R
```

4.1.13 Privacy Mode Algorithm

Privacy Mode allows meeting participants to temporarily pause transcription and recording during sensitive discussions. When toggled on, the system stops processing incoming audio entirely: no speech is stored, transcribed, or forwarded. Each pause is recorded as a timestamped interval, and when AI insights are generated after the meeting, any utterances that fall within those intervals are excluded automatically.

Algorithm 13 Privacy Mode: Transcription Suppression and Insight Filtering

Require: Meeting ID m_{id}, audio chunk c, toggle $\delta \in \{\texttt{enable}, \texttt{disable}\}$, transcript $\mathcal{T}$

Ensure: Chunks suppressed during privacy windows; AI insights exclude privacy-interval utterances

1: $\sigma \leftarrow$ CacheGet(m_{id}) {Phase 1: State Retrieval}
2: **if** $\sigma = \texttt{null}$ **then**
3: $\quad\sigma \leftarrow$ QueryDatabase(m_{id}, `metadata.privacyMode`); CacheSet(m_{id}, σ)
4: **end if**
5: {Phase 2: State Transition}
6: **if** $\delta = \texttt{enable}$ **and** $\neg\sigma.\mathit{enabled}$ **then**
7: $\quad\sigma \leftarrow \{\mathit{enabled} : \texttt{true},\ \mathit{intervalStart} : \texttt{CurrentTimestamp()}\}$
8: $\quad$PersistAndInvalidate(m_{id}, σ)
9: $\quad$BroadcastPrivacyEvent(m_{id}, `privacy_on`); AppendSystemLine(m_{id}, `"Privacy Mode ON"`)
10: **else if** $\delta = \texttt{disable}$ **and** $\sigma.\mathit{enabled}$ **then**
11: $\quad$AppendPrivacyInterval(m_{id}, $\{\mathit{start} : \sigma.\mathit{intervalStart},\ \mathit{end} : \texttt{CurrentTimestamp()}\}$)
12: $\quad\sigma \leftarrow \{\mathit{enabled} : \texttt{false},\ \mathit{intervalStart} : \texttt{null}\}$
13: $\quad$PersistAndInvalidate(m_{id}, σ)
14: $\quad$BroadcastPrivacyEvent(m_{id}, `privacy_off`); AppendSystemLine(m_{id}, `"Privacy Mode OFF"`)
15: **end if**
16: {Phase 3: Real-Time Audio Suppression}
17: **if** $\sigma.\mathit{enabled}$ **then**
18: $\quad$**return** {Chunk silently dropped; not forwarded to TranscriptionService or AudioRecorder}
19: **else**
20: $\quad$ForwardToTranscriptionPipeline(c); ForwardToAudioRecorder(c)
21: **end if**
22: $\Pi \leftarrow$ LoadPrivacyIntervals(m_{id}) {Phase 4: Retrospective Insight Filtering}
23: $\mathcal{T}_{filtered} \leftarrow \{u \in \mathcal{T} \mid \nexists\, \pi \in \Pi : u.\mathit{timestamp} \in [\pi.\mathit{start},\ \pi.\mathit{end}]\}$
24: GenerateAIInsights($\mathcal{T}_{filtered}$)

4.1.14 Calendar Integration Algorithm

Calendar integration synchronizes external calendar events with Kairo's meeting records. The system supports OAuth-based integration (Google/Microsoft) as the primary method, with fallback support for iCalendar (ICS) feeds and CalDAV for users or organizations unable to use OAuth. A scheduled synchronization job periodically fetches upcoming events, maps them to internal meeting records, and upserts them using stable external identifiers to maintain consistency with automated workflows such as bot joining and reminders.

Algorithm 14 Calendar Integration and Event Synchronization

Require: User u; calendar connections $\mathcal{C}$; time window $[t_{start}, t_{end}]$; meeting set $\mathcal{M}$
Ensure: Events synchronized to $\mathcal{M}$; no duplicate processing; provider tokens valid

```
Phase 1: OAuth Token Management
for each OAuth connection c ∈ C with c.type = OAuth do
    Load encrypted token material: τ ← LoadTokens(c)
    if access token expired then
        Refresh token: τ' ← RefreshToken(c, τ); Save(τ')
        if refresh failed then
            Mark connection error; continue
        end if
    end if
end for
Phase 2: Event Fetching (OAuth)
for each OAuth connection c do
    E_OAuth ← ListEventsViaOAuth(c, [t_start, t_end])
    for each event e ∈ E_OAuth do
        m ← MapToMeeting(e, c)
        UpsertMeeting(M, m) {idempotent on stable keys}
    end for
end for
Phase 3: Reconciliation
for each connection c do
    ReconcileCancellations(M, c) {remove cancelled events from Kairo}
end for
return Updated meeting set M
```

4.2 External APIs/SDKs

The following table describes all third-party APIs and SDKs integrated into the Kairo system, their purposes, and specific implementations.

API/SDK and Version	Description	Purpose of Usage	API Endpoint/-Function/Class Used
Supabase SDK v2	Backend-as-a-Service platform	PostgreSQL database management, authentication, and file storage	`supabase.auth.*`, `supabase.storage.*`, real-time subscriptions
pgvector	PostgreSQL extension for vector similarity search	Indexing and querying semantic embeddings for contextual retrieval and related meeting detection	`CREATE EXTENSION vector`, HNSW index, cosine similarity queries via Prisma raw SQL
Hugging Face Inference API	Hosted inference API for open-source large language models	LLM-based text generation, summarization, and NLP agent tasks	`POST /models/{model}`, `hf_infer()`, Bearer token auth
GroqCloud API (Llama 3 / Mixtral)	High-performance inference API for open-source LLMs	Abstractive meeting summarization, decision extraction, sentiment analysis, and action item generation	`/summarizationService`, model: `llama3`, `mixtral`
WhisperX / Faster-Whisper	Optimised Whisper-based speech recognition with diarization	Real-time and post-meeting transcription with speaker attribution	`whisperx.load_model()`, `model.transcribe()`, `DiarizationPipeline`
Pyannote.audio v3	Speaker diarization library	Identifying and segmenting speakers in meeting audio	`Pipeline.from_pretrained()`, `pipeline(audio_file)`

API/SDK and Version	Description	Purpose of Usage	API Endpoint/-Function/Class Used
Puppeteer v21 + puppeteer-extra-plugin-stealth	Headless browser automation library	Automating bot entry into Google Meet and Zoom web clients for audio capture	`puppeteer.launch()`, `page.goto()`, `page.evaluate()`
Prisma ORM v6	Type-safe database ORM for Node.js	Database schema management, migrations, and query abstraction over PostgreSQL	`PrismaClient`, `prisma.meeting.*`, `prisma.$queryRaw()`
React v19	JavaScript UI library	Building responsive, interactive user interfaces for the web dashboard	Components, hooks (`useState`, `useEffect`), context API
Tailwind CSS v3	Utility-first CSS framework	Styling UI components with responsive design	Utility classes, custom configurations
WebSocket (ws v8)	Real-time communication library	Broadcasting live transcription updates and micro-recap notifications to connected clients	`WebSocketServer`, `ws.send()`, `ws.on()`

Table 4.2: External APIs and SDKs Used in Kairo Implementation

4.3 Testing Details

Comprehensive testing was performed throughout the development lifecycle to ensure system correctness, reliability, and performance. The testing strategy encompassed multiple levels, from individual unit tests to complete end-to-end workflow validation.

4.3.1 Unit Testing

Unit tests were implemented for all critical system components to verify that individual functions and methods operate correctly in isolation. Each unit test focuses on a specific functionality and validates both expected behaviour and edge cases.

Automated testing was conducted across three layers of the system. The Python AI layer was tested using **pytest**, covering individual insight agents (summarisation, decision extraction, action item generation, sentiment analysis, and participant analysis) as well as the full parallel orchestrator workflow, totalling 31 automated test cases. The Node.js backend service layer was tested using **Mocha/Chai**, validating service-level logic including `PrivacyModeService`, `TranscriptionService`, `AIInsightsService`, and meeting lifecycle state transitions across 93 automated test cases. Backend and frontend REST API endpoints were validated using **Postman**, covering all API routes including authentication, meeting management, transcription control, privacy mode toggling, and semantic search. These automated suites were complemented by extensive manual end-to-end testing covering the complete user journey from sign-up through meeting capture to post-meeting intelligence pipeline delivery.

4.3.1.1 Authentication Module Test Cases

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC001	Verify user registration with valid credentials	Database is accessible and email does not exist	1. Navigate to registration page 2. Enter valid email 3. Enter strong password 4. Click Register	Email: test@kairo. Password: SecurePass12	User account created successfully, confirmation email sent	User account was created successfully with ID 1001, confirmation email was sent and delivered to test@kair within 2 seconds	New user record exists in database	Pass
TC002	Verify registration fails with duplicate email	User with email already exists in database	1. Navigate to registration 2. Enter existing email 3. Enter password 4. Click Register	Email: existing@kairo. Password: Pass123!	Error message: "Email already registered"	System returned HTTP 409 error, error message "Email already registered" was displayed on UI, no new database entry was created	No duplicate user created	Pass

Continued on next page

Table 4.3 – continued from previous page

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC003	Verify login with valid credentials	User account exists and is active	1. Navigate to login page 2. Enter registered email 3. Enter correct password 4. Click Login	Email: user@kairo Password: User-Pass123!	User redirected to dashboard, session token generated	User was logged in and redirected to /dashboard, JWT token was generated with 24h expiration, session was stored in browser localStorage	User session active until logout	Pass
TC004	Verify login fails with invalid password	User account exists	1. Navigate to login 2. Enter valid email 3. Enter wrong password 4. Click Login	Email: user@kairo Password: Wrong-Pass	Error message: "Invalid credentials"	System returned HTTP 401 error, "Invalid credentials" message was displayed, login attempt was logged in audit table, no session was created	User not logged in, no session created	Pass

Continued on next page

Table 4.3 – continued from previous page

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC005	Verify password strength validation	Registration form is loaded	1. Enter weak password (less than 8 chars) 2. Attempt to register	Password: 123	Error: "Password must be at least 8 characters"	Real-time validation was triggered, error message appeared below password field, Register button remained disabled until valid password entered	User not registered	Pass

Table 4.3: Authentication Module Test Cases

4.3.1.2 Meeting Management Test Cases

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC006	Verify meeting creation with valid data	User is logged in and has workspace access	1. Click Create Meeting 2. Enter meeting title 3. Select date/time 4. Add participants 5. Click Save	Title: "Q4 Planning" Date: 2024-12-15 Time: 10:00 AM Participants: 3 users	Meeting created and appears in calendar	Meeting was created with ID meeting_789, stored in database with status "scheduled", calendar event was synced successfully, email invitations were sent to all 3 participants	Meeting stored in database with status "scheduled"	Pass

Continued on next page

Table 4.4 – continued from previous page

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC007	Verify Kairo Bot joins meeting automatically	Meeting is scheduled and calendar integration active	1. Start scheduled meeting 2. Wait for bot join trigger	Meeting ID: meeting_123 Auto-join: enabled	Bot joins within 30 seconds of meeting start	Kairo Bot joined the meeting 18 seconds after meeting started, bot status was updated to "joined", audio recording was initiated successfully	Bot status: "joined", recording initiated	Pass
TC008	Verify manual bot invitation	Meeting in progress without bot	1. Click "Invite Kairo Bot" 2. Confirm invitation	Meeting link: zoom.us/j/1	Bot joins meeting within 1 minute	Bot connection was established in 42 seconds, recording and real-time transcription became active, bot appeared in participant list	Bot actively recording and transcribing	Pass

Continued on next page

Table 4.4 – continued from previous page

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC009	Verify meeting cancellation	Meeting is scheduled but not started	1. Select meeting 2. Click Cancel Meeting 3. Confirm cancellation	Meeting ID: meeting_456	Meeting status changed to "cancelled", participants notified	Meeting status was updated to "cancelled" in database, cancellation emails were sent to all 4 participants, calendar event was removed, bot join trigger was disabled	No bot join attempted, database updated	Pass

Table 4.4: Meeting Management Test Cases

4.3.1.3 Transcription Service Test Cases

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC010	Verify real-time transcription accuracy	Bot has joined meeting and audio stream is active	1. Speak test phrase 2. Wait for transcription 3. Verify text accuracy	Audio: "Let's discuss the project timeline"	Transcription matches spoken words with >90% accuracy	Transcript was generated as "Let's discuss the project timeline" with 96% word accuracy, latency was 2.3 seconds, timestamp was stored	Transcript saved with timestamp	Pass
TC011	Verify speaker identification	Meeting has multiple participants	1. Different speakers talk 2. Check speaker labels in transcript	3 speakers with distinct voices	Each transcript segment correctly labeled with speaker ID	Speakers 1, 2, and 3 were correctly identified across all 15 segments, speaker labels were stored with confidence scores averaging 92%	Speaker labels stored in database	Pass

Continued on next page

Table 4.5 – continued from previous page

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC012	Verify transcription with background noise	Audio contains noise (typing, background conversations)	1. Capture noisy audio 2. Process transcription	Audio with 60dB background noise	Primary speech transcribed, noise filtered out	Primary speech was transcribed at 89% accuracy, noise cancellation reduced background by 45dB, clean transcript was generated	Clean transcript generated	Pass
TC013	Verify handling of audio interruptions	Network interruption during meeting	1. Simulate network drop 2. Reconnect after 10 seconds 3. Check transcript continuity	10-second gap in audio	Transcript shows gap marker, resumes after reconnection	Transcript contained [Audio Gap: 10s] marker, bot reconnected in 3 seconds after network was restored, transcription resumed with 1-second buffer overlap	No data loss, gap documented	Pass

Table 4.5: Transcription Service Test Cases

4.3.1.4 Action Item Extraction Test Cases

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC014	Verify action item detection from transcript	Meeting transcript is complete	1. Run action item extraction 2. Review detected items	Transcript: "John will complete the report by Friday"	Action item created: "Complete report", assigned to John, due Friday	Action item AI_001 was created with title "Complete report", was assigned to John (user_456 deadline was set to Friday 2024-12-13 17:00, stored in database	Action items stored in database	Pass
TC015	Verify deadline extraction	Transcript contains temporal references	1. Process text with deadlines 2. Verify extracted dates	Text: "Submit by next Monday", "Due end of month"	Deadlines: specific date for Monday, last day of current month	Deadlines were extracted as 2024-12-09 (Monday) and 2024-12-31 (end of month), both were converted to ISO 8601 format	Deadlines in ISO format	Pass

Continued on next page

Table 4.6 – continued from previous page

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC016	Verify assignee identification	Transcript mentions participant names	1. Extract action with assignee 2. Match to participant list	"Sarah should review the document"	Assignee: Sarah (matched to participant ID)	Sarah was matched to user_789 in participants list with 95% confidence, task was assigned and notification was sent	Task assigned to correct user	Pass
TC017	Verify priority estimation	Action items with urgency indicators	1. Analyze items for priority 2. Assign priority levels	"URGENT: Fix bug", "Consider updating logo"	High priority for urgent item, low for consideration	Bug fix was assigned priority: High (urgency keyword detected), logo update was assigned priority: Low, priority values were stored	Priority levels stored	Pass

Continued on next page

Table 4.6 – continued from previous page

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC018	Verify handling of ambiguous tasks	Transcript without clear action items	1. Process general discussion transcript 2. Check for false positives	Transcript: "We might think about this later"	No action item created (confidence too low)	Action item confidence score was 32% (below 60% threshold), no task was created, logged as low-confidence mention	No spurious tasks generated	Pass

Table 4.6: Action Item Extraction Test Cases

4.3.1.5 Whisper Mode Test Cases

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC019	Verify micro-recap broadcast at configured interval	Meeting is in progress with active transcript buffer; WebSocket clients connected	1. Start meeting 2. Wait for recap interval ($\Delta t = 5$ min) 3. Verify recap broadcast to all clients	Meeting: 4 connected clients; recap interval: 5 minutes	Micro-recap generated (max 3 sentences) and broadcast to all 4 clients via WebSocket	Recap was generated after 5 minutes with 3 sentences summarising discussion; broadcast latency was 340ms; recap persisted in database with spanStart and spanEnd timestamps; all 4 clients received the message	Recap stored in database; clients display notification	Pass

Continued on next page

Table 4.7 – continued from previous page

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC020	Verify catch-up recap for late-joining participant	Meeting in progress for 7 minutes; new participant joins	1. Participant joins meeting at 7-minute mark 2. Trigger catch-up recap 3. Verify recap covers full meeting span	New participant: user_321; joined at 00:07:00	Catch-up recap generated covering 00:00:00–00:07:00 and sent only to new participant	Catch-up recap was generated in 1.8 seconds; covered the full prior 7 minutes; was sent exclusively to user_321; other participants did not receive a duplicate	Only late participant receives catch-up recap	Pass
TC021	Verify no recap generated when transcript buffer is empty	Meeting in progress but no speech detected for recap interval period	1. Allow meeting to run silently for interval duration 2. Check that no recap is broadcast	Empty audio period: 5 minutes	No recap broadcast; loop continues without error	System detected empty segment list; **continue** branch was followed; no WebSocket message sent; no empty recap persisted in database	No spurious recaps generated	Pass

Table 4.7: Whisper Mode Test Cases

4.3.1.6 Meeting Memory Graph Test Cases

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC022	Verify Meeting node and attendee edges created after processing	Post-meeting pipeline has completed Phase 8; approved insights available	1. Complete a meeting with 3 attendees 2. Run knowledge graph update 3. Inspect graph for Meeting node and Participant edges	Meeting ID: meeting_999; attendees: user_1, user_2, user_3	1 Meeting node created; 3 Participant nodes linked via `attended` edges	Meeting node (meeting_999) was upserted in graph; 3 Participant nodes were linked with `attended` edges; all nodes persisted via `PersistG`	Graph contains correct node and edge count	Pass
TC023	Verify cross-meeting `semantical` edge created for similar prior meeting	Vector store contains at least one prior meeting with similar topic embeddings	1. Process new meeting on same topic as prior meeting 2. Run `FindSimilarMe` 3. Verify edge created if score $\geq \theta_{relate}$	New meeting: budget discussion; prior meeting: meeting_500 (budget review); similarity score: 0.88; $\theta_{relate} = 0.75$	`semantical` edge created between meeting_999 and meeting_500	Similarity score 0.88 exceeded threshold 0.75; bidirectional `semantic` edge was upserted with weight 0.88; `shared_p` edge also created (2 shared attendees)	Two cross-meeting relationship edges persisted	Pass

Continued on next page

Table 4.8 – continued from previous page

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC024	Verify `followed_up` edge created when prior task is referenced in new transcript	Prior task node (task_101) exists in graph; new transcript references it by description	1. Process new meeting transcript that mentions task_101 2. Run `TaskMentioned` check 3. Verify `followed_up` edge	Prior task: "Prepare Q4 budget report" (task_101); new transcript: "John confirmed the Q4 budget report is complete"	`followed_up` edge from new meeting node to task_101	`TaskMent` returned true for task_101; `followed` edge was upserted from meeting_999 to task_101; graph delta was persisted	Follow-up relationship visible in knowledge graph	Pass

Table 4.8: Meeting Memory Graph Test Cases

4.3.1.7 Semantic Search Test Cases

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC025	Verify semantic search returns relevant results for natural language query	Vector store contains embeddings for at least 5 meetings; user has access to all	1. Enter query: "budget planning discussion" 2. Submit search 3. Inspect ranked results	Query: "budget planning discussion"; workspace with 5 meetings; 2 budget-related meetings in store	Budget-related meetings ranked in top 2 results; results grouped by content type with highlight snippets	Embeddin was generated for query; pgvector cosine search returned 2 budget meetings in top 2 positions (scores: 0.91, 0.84); results were enriched with highlight snippets and breadcrumbs; grouped into Transcript and Summary content types	Correct meetings surfaced with relevance scores	Pass

Continued on next page

Table 4.9 – continued from previous page

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC026	Verify RBAC enforcement in semantic search scope	Workspace has 5 meetings; user has access to only 3	1. Execute semantic search as restricted user 2. Verify results contain only accessible meetings	User: user_limite (access to meetings 1, 2, 3 only); query: "project timeline"	Results contain only meetings within user's access scope; meetings 4 and 5 not returned	`GetAcces` returned scope of 3 meetings; pgvector and full-text search were both scoped to those 3 meetings; meetings 4 and 5 did not appear in any result set	No unauthorised meeting data leaked	Pass

Continued on next page

Table 4.9 – continued from previous page

ID	Objective	Pre-condition	Steps	Test Data	Expected Result	Post-Condition	Actual Result	Pass/Fail
TC027	Verify hybrid retrieval fuses vector and keyword results correctly	Vector store and full-text index both populated with test data	1. Submit query that has both semantic and keyword matches 2. Inspect fusion scoring	Query: "action items for Sarah"; vector match: meeting_200 (score: 0.78); keyword match: meeting_300 (exact name "Sarah")	Both meetings appear in results; fusion score $\alpha \cdot s_{vec} + \beta \cdot s_{kw} + \gamma \cdot$ RecencyBoos applied	meeting_2 and meeting_300 both retrieved; deduplication step confirmed no overlap; fusion scores were computed (0.81 and 0.76 respectively); meeting_200 ranked first due to higher combined score	Ranked, fused result list returned	Pass

Table 4.9: Semantic Search Test Cases

4.3.2 Integration Testing

Integration tests verify that multiple system components work together correctly. These tests validate data flow between modules, API integrations, and database interactions.

4.3.2.1 End-to-End Workflow Test

Field	Details
Test Case ID	TC028
Objective	Verify complete meeting lifecycle from scheduling through post-meeting intelligence pipeline, including knowledge graph and vector store updates
Pre-condition	User logged in with workspace configured; pgvector extension enabled in PostgreSQL
Test Steps	1. Schedule meeting with 4 participants via calendar 2. Kairo Bot auto-joins at scheduled time 3. Real-time transcription and speaker diarization active throughout 4. Whisper Mode micro-recaps broadcast at 5-minute intervals 5. Meeting ends; post-meeting pipeline triggered automatically 6. Full diarized transcript generated from merged audio 7. AI agents run in parallel: summary, decisions, action items, sentiment, participant analysis 8. Action items materialised as Kanban tasks 9. Vector store updated with transcript chunks and insights embeddings 10. Meeting Memory Graph updated with nodes, edges, and cross-meeting relationships 11. Semantic search queried to verify new meeting content is retrievable
Test Data	Full meeting: 30 min duration, 4 participants, 3 agenda topics, 5 expected action items; 2 prior related meetings in vector store for cross-meeting linking
Expected Result	All pipeline phases complete successfully: transcript $\geq$95% accurate, summary generated within 10 seconds, 5 action items extracted and assigned, Kanban updated, vector embeddings stored, knowledge graph contains new Meeting node with typed edges, semantic search returns new meeting in top-3 results for relevant query
Actual Result	Meeting was scheduled (ID: meeting_999) and bot joined in 20 seconds. Real-time transcript was generated with 94% accuracy (1,847 words) across all 4 speakers. Three Whisper Mode micro-recaps were broadcast at 5, 10, and 15-minute marks. Post-meeting pipeline completed all 9 phases: audio was finalised and persisted, full diarized transcript was aligned with speaker profiles, 5 AI agents completed in parallel (avg. 6.2 seconds). All 5 action items were materialised as Kanban cards in the "To Do" column. Vector store was updated with 23 transcript chunk embeddings and 5 insight embeddings. Knowledge graph was updated with 1 Meeting node, 4 Participant edges, 3 Topic nodes, 2 Decision nodes, and 5 Task nodes; 2 cross-meeting `semantically_related` edges were created to prior meetings. Semantic search for "Q4 budget discussion" returned the new meeting as the top result with a relevance score of 0.91
Post-Condition	Complete 9-phase data pipeline successful; all artefacts persisted in PostgreSQL; knowledge graph and vector store reflect new meeting; workspace members notified

Continued on next page

Table 4.10 – continued from previous page

Field	Details
Status	Pass

Table 4.10: Integration Test Case: Complete Meeting Lifecycle

4.3.3 Testing Statistics Overview

The following table summarises the complete testing coverage achieved across all automated and manual testing activities conducted throughout the development of the complete Kairo platform across all four iterations.

Tool / Method	Scope of Coverage	Total Tests	Outcome
Postman	Validation of all backend and frontend REST API endpoints, including authentication, meeting management, transcription control, privacy mode, and semantic search	All API routes	All endpoints returned expected HTTP responses and payloads
pytest	AI insight agents individually (summarisation, decision extraction, action item generation, sentiment analysis, participant analysis) and the full parallel orchestrator workflow	31	All 31 tests passed
Mocha / Chai	Backend JavaScript service-layer logic including `PrivacyModeService`, `TranscriptionService`, `AIInsightsService`, and meeting lifecycle state transitions	159	All 159 tests passed
Manual End-to-End	Complete user workflow: Sign-up → workspace creation → meeting scheduling → bot join → real-time transcription → Whisper Mode → post-meeting pipeline → knowledge graph → semantic search	$\geq$400	All critical paths validated; no blocking defects identified

Table 4.11: Testing Statistics Overview — Kairo FYP (Complete Development)

```
PS A:\FYP\Kairo-webapp\backend> npm test
    ✓ should accurately calculate MD5 checksums of uploads to verify integrity
    ✓ should extract exact audio durations natively using ffprobe or magic bytes before upload
    ✓ should reject completely empty buffers (0-byte uploads) with a 400 Bad Request
    ✓ should assign strict private ACL rules to uploaded meeting recordings
    ✓ should automatically invoke a background deletion worker for temporary audio segments
    ✓ should securely delete an asset from the Cloud bucket requested by authorized owner
    ✓ should throw an HTTP 403 Forbidden if a non-owner tries to delete bucket assets

  WebSocketServer
    ✓ should successfully establish a Socket.io connection with a client
    ✓ should enforce JWT token Authentication before allowing room joins
    ✓ should firmly reject WebSocket connections without valid Handshake headers
    ✓ should allow users to join their specific Workspace rooms (e.g., "workspace_5")
    ✓ should allow live meeting participants to join a specific "meeting_101" room
    ✓ should correctly Broadcast "whisper_recap" events only to the correct meeting room
    ✓ should broadcast "new_transcript_chunk" to users watching a live meeting in progress
    ✓ should push Kanban board updates ("task_moved") to all active workspace users immediately
    ✓ should correctly handle client disconnects and clean up mapped user memory
    ✓ should map socket IDs to User IDs successfully for direct messaging
    ✓ should keep connections alive efficiently with automated heartbeat ping/pongs
    ✓ should limit standard users from broadcasting global system events
    ✓ should successfully broadcast Meeting Status updates (e.g. "Meeting Ended")
    ✓ should prevent Cross-Site-WebSocket-Hijacking using strict CORS origins
    ✓ should gracefully reconnect clients during short network dropouts
    ✓ should send generic error payloads formatting standard UI popups

  Whisper Mode (Live Micro-Recaps)
    MicroSummaryService
      ✓ should generate a live micro-recap from recent 5-minute transcript chunks
      ✓ should broadcast the real-time recap to all active WebSocket clients in the meeting room
      ✓ should skip generating recaps if the transcript chunk hasn't changed significantly
      ✓ should respect the WHISPER_MODE_ENABLED environment variable gating
      ✓ should handle manual /whisper/trigger REST calls perfectly
      ✓ should correctly identify sudden topic shifts during mid-meeting
      ✓ should attach a precise UTC timestamp to every generated recap
      ✓ should format the recap in 2-3 concise sentences maximum
      ✓ should drop duplicate recaps if the AI repeats itself
      ✓ should prevent WebSocket event flooding by debouncing broadcasts
      ✓ should recover gracefully if the Groq LLM times out mid-generation
      ✓ should format bullet points nicely for frontend UI consumption
      ✓ should handle extremely fast speech with overlapping segments
      ✓ should securely authenticate the WebSocket connection before pushing
      ✓ should clean up old recap caches from memory when the meeting ends
      ✓ should track token usage efficiently to minimize Groq API costs


  159 passing (337ms)
```

Figure 4.1: Complete Mocha/Chai Test Suite Results (159 Tests)

Chapter 5

Conclusions and Future Work

5.1 Conclusion

This thesis presented the design and implementation of Kairo across all iterations, an intelligent meeting collaboration platform that transforms unstructured discussions into structured, actionable knowledge. Kairo addresses key limitations of conventional meeting tools by enabling persistent memory, cross-meeting continuity, and automated extraction and retrieval of decisions, tasks, and insights.

Throughout all completed iterations, Kairo delivers a comprehensive end-to-end workflow covering authentication and role-based access control, workspace and team management, meeting scheduling, automatic bot participation in online meetings, real-time transcription with speaker diarization, action item extraction, and AI-driven meeting summarization. The platform further supports interactive transcript review (including timeline-based navigation), in-meeting note-taking, and an analytics dashboard for meeting and participant insights. In addition, a Privacy Mode mechanism was implemented to pause and resume transcription and ensure that sensitive segments are excluded from downstream AI processing. Overall, the system follows a scalable three-tier architecture, providing a robust and maintainable foundation for intelligent meeting intelligence and organizational knowledge retention. Kairo has proven to work beyond expectations, delivering a production-ready platform that successfully transforms how organizations manage and extract value from their meetings.

5.2 Future Work

While Kairo has been successfully developed and delivered with all core functionalities completed and validated, several promising avenues remain for future enhancement and

expansion:

- **External Tool Integration:** Integration with external enterprise tools such as Jira to enable seamless synchronization of action items and real-time collaboration. This will allow teams to maintain consistent workflows across their existing tool ecosystem while leveraging Kairo's intelligent meeting intelligence capabilities.
- **Expansion to Microsoft Teams:** Comprehensive expansion of Kairo's capabilities to Microsoft Teams, enabling deeper integration and native support for Teams-based workflows and collaboration patterns.

These enhancements will be prioritized based on user feedback and organizational needs, further solidifying Kairo's position as a comprehensive platform for intelligent meeting collaboration and organizational knowledge management.

Bibliography

[1] M. Bain. WhisperX — word-level timestamps and speaker diarization for whisper. GitHub Repository, 2024.

[2] Santosh Bhupathi. Building scalable AI-powered applications with cloud databases: Architectures, best practices and performance considerations. *arXiv preprint arXiv:2504.18793*, April 2025.

[3] X. Chen, L. Tankelevitch, R. Vanukuru, A. E. Scott, P. Panda, and S. Rintel. Are we on track? AI-assisted active and passive goal reflection during meetings. *arXiv preprint arXiv:2504.01082*, April 2025.

[4] X. Chen, N. Yap, X. Lu, A. Gunal, and X. Wang. MeetMap: Real-time collaborative dialogue mapping with LLMs in online meetings. *arXiv preprint arXiv:2502.01564*, February 2025.

[5] EnterpriseDB (EDB). Postgres: The future-proof data platform for the enterprise. EDB Enterprise Blog, 2024. Discusses Postgres' AI and analytics readiness.

[6] Fathom AI Team. Fathom AI notetaker. https://fathom.video, January 2024.

[7] Fireflies.ai Team. Fireflies.ai meeting assistant. https://fireflies.ai, January 2024.

[8] L. Golia and J. Kalita. Action-item-driven summarization of long meeting transcripts. In *Proceedings of the 7th International Conference on NLPIR*, Seoul, December 2023.

[9] JuanMC et al. Diart — real-time diarization framework. GitHub, 2024.

[10] Krisp AI Team. Krisp AI meeting assistant. https://krisp.ai, January 2024.

[11] J. Liu, C. Deng, Q. Zhang, Q. Chen, and W. Wang. Meeting action item detection with regularized context modeling. *arXiv preprint arXiv:2303.16763*, March 2023.

[12] MeetGeek AI Team. MeetGeek AI meeting assistant. https://meetgeek.ai, January 2024.

[13] Motion AI Team. Motion AI productivity assistant. https://usemotion.com, January 2024.

[14] Neo4j Graph Database Docs. Knowledge graphs and graph-based retrieval for conversational memory, 2024.

[15] Otter.ai Team. Otter.ai real-time transcription. https://otter.ai, January 2024.

[16] Recall.ai. How can I create a zoom bot that joins meetings and interacts as a participant? Recall.ai Blog, 2024.

[17] Resemble.ai. Resemblyzer — voice encoder for speaker embeddings. GitHub Package, 2023.

[18] Y. Weng and J. Wu. Database management systems for artificial intelligence: Comparative analysis of PostgreSQL and MongoDB. *World Journal of Advanced Research and Reviews*, 2025. Analysis of PostgreSQL vs MongoDB for AI.